开发者社区

DTK

# DTK 26.10
# RCCL 库使用手册

DTK-261000-024-0915-939

# 目　录

1 RCCL 库简介 ..... 1
2 环境要求 ..... 1
3 核心概念 ..... 1
3.1 Communicator 管理 ..... 1
3.2 通信原语 ..... 2
3.3 简单 demo ..... 5
3.4 细粒度通信 API 使用说明（DTK 25.04–26.04 适用） ..... 15
3.4.1 细粒度集合通信 API 使用 ..... 15
3.4.2 细粒度点对点通信 API 使用 ..... 16
4 API 详解 ..... 17
4.1 通讯组创建及管理类函数 ..... 17
4.1.1 ncclGetLastError ..... 17
4.1.2 ncclGetVersion ..... 17
4.1.3 ncclGetUniqueId ..... 17
4.1.4 ncclCommInitRank ..... 18
4.1.5 ncclCommInitAll ..... 18
4.1.6 ncclCommInitRankConfig ..... 19
4.1.7 ncclCommFinalize ..... 20
4.1.8 ncclCommDestroy ..... 20
4.1.9 ncclCommAbort ..... 20
4.1.10 ncclCommGetAsyncError ..... 21
4.1.11 ncclCommCount ..... 21
4.1.12 ncclCommCuDevice ..... 21
4.1.13 ncclCommUserRank ..... 22
4.1.14 ncclCommShrink ..... 22
4.1.15 ncclCommInitRankScalable ..... 23
4.2 集合通讯函数 ..... 23
4.2.1 ncclAllReduce ..... 23

开发者社区

4.2.2 ncclBroadcast ..... 24
4.2.3 ncclReduce ..... 25
4.2.4 ncclAllGather ..... 26
4.2.5 ncclReduceScatter ..... 26
4.2.6 ncclAllReduceWithBias ..... 27
4.3 组调用函数 ..... 28
4.3.1 ncclGroupStart ..... 28
4.3.2 ncclGroupEnd ..... 28
4.3.3 ncclGroupSimulateEnd ..... 28
4.4 点对点通讯函数 ..... 29
4.4.1 ncclSend ..... 29
4.4.2 ncclRecv ..... 30
4.5 数据类型 ..... 31
4.5.1 ncclComm_t ..... 31
4.5.2 ncclResult_t ..... 31
4.5.3 ncclDataType_t ..... 32
4.5.4 ncclRedOp_t ..... 32
4.5.5 ncclScalarResidence_t ..... 33
4.5.6 ncclConfig_t ..... 33
4.5.7 ncclSimInfo_t ..... 34
4.6 自定义归约操作符 ..... 35
4.6.1 ncclRedOpCreatePreMulSum ..... 35
4.6.2 ncclRedOpDestroy ..... 36
4.7 细粒度通信接口（DTK 25.04–26.04 适用） ..... 37
4.7.1 duNcclAllreduce ..... 37
4.7.2 duNcclAllgather ..... 37
4.7.3 duNcclBcast ..... 38
4.7.4 duNcclBroadcast ..... 39
4.7.5 duNcclReduceScatter ..... 40
4.7.6 duNcclReduce ..... 41

开发者社区

4.7.7 duNcclSend ..... 42
4.7.8 duNcclRecv ..... 43
4.7.9 duNcclGather ..... 44
4.7.10 duNcclScatter ..... 45
4.7.11 duNcclAllToAll ..... 46
4.7.12 duNcclAllToAllv ..... 47
4.8 内存管理 ..... 48
4.8.1 ncclMemAlloc ..... 48
4.8.2 ncclMemFree ..... 49
4.8.3 ncclCommRegister ..... 49
4.8.4 ncclCommDeregister ..... 50
4.8.5 ncclCommWindowRegister ..... 51
4.8.6 ncclCommWindowDeregister ..... 52
5 性能调优 ..... 52
5.1 拓扑感知配置 ..... 52
5.2 拓扑文件生成 ..... 53
5.3 环境变量优化 ..... 54
5.4 通道数、算法、通信协议配置 ..... 60
5.5 超节点支持 ..... 61
6 调试与诊断 ..... 62
6.1 日志分析 ..... 62
6.1.1 保存日志进行确认 ..... 62
6.1.2 报错问题日志分析 ..... 62
6.2 hipprof 分析 ..... 63
开发者社区

## 版本更新：

- DTK-25.04.1：
  - 创建文档。

开发者社区

# 1 RCCL 库简介

RCCL 是 DTK 中的一套 GPU 集合通信库，支持以下通信原语：all-reduce, all-gather, reduce, broadcast, reduce-scatter, gather, scatter, all-to-all，该库同时支持 GPU 间直接点对点通信，RCCL 可在多种硬件架构下实现高带宽通信，包括 PCIe 互联架构、Hylink 高速直连、基于 InfiniBand Verbs 或 TCP/IP 套接字的网络架构。RCCL 库核心特性：支持单节点与多节点多个 GPU 部署，兼容单进程与多进程应用场景，采用环形与树形算法实现集合操作，针对吞吐量与时延进行双重优化。

# 2 环境要求

硬件要求：DCU 以及配套驱动，多 DCU 系统；

软件栈要求：DTK（优先使用高版本 DTK）、MPI 库；

运行环境配置：多节点环境要求每个机器的配置完全相同，DCU 可见性，可通过 HIP_VISIBLE_DEVICES 设置。

# 3 核心概念

## 3.1 Communicator 管理

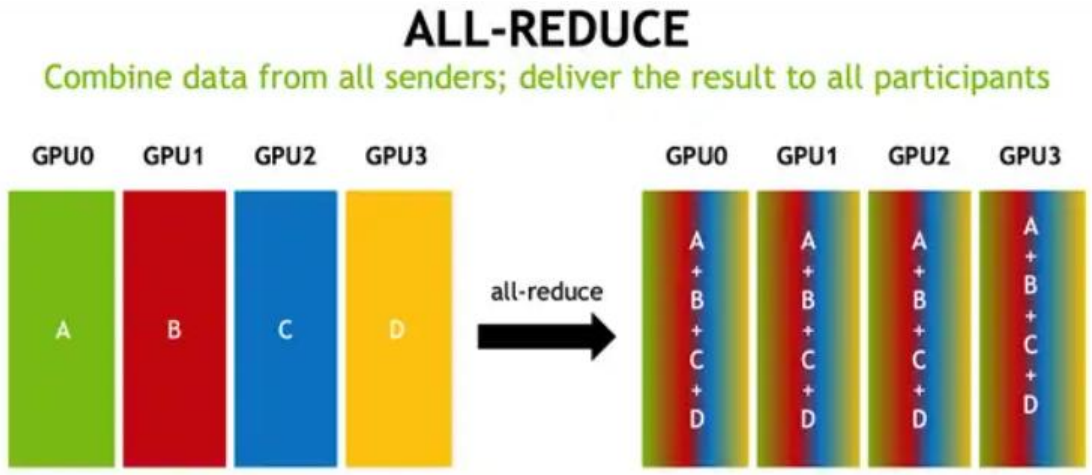
RCCL 初始化流程：

1. 构建 Bootstrap 网络：将通信域内所有 rank 连接成一个逻辑环状结构；
2. 拓扑感知：通信域内的每个 rank 探测其所在节点的硬件拓扑;
3. 探索通信路径：构建 ring 图、tree 图、collnet 图等，通信域完成封装通信策略；
4. 建立数据传输链路：为即将开始的实际数据传输选择合适的传输链路，通信域管理和持有资源；
5. 全局信息同步：通信域内所有成员对组的状态和配置达成一致共识。
6. RCCL 通信流程：
7. 完成初始化之后调用通信算子的 API ncclAllreduce 等，根据拓扑信息进行通信算法、协议评估，构建核函数启动参数，调用通信算子对应的设备函数完成通信

开发者社区

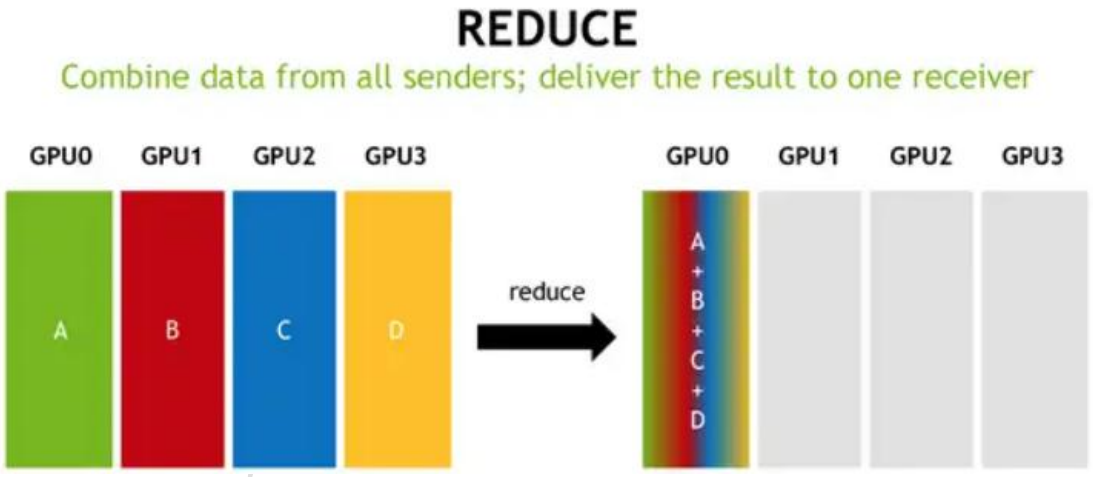
## 3.2 通信原语

1) All-Reduce



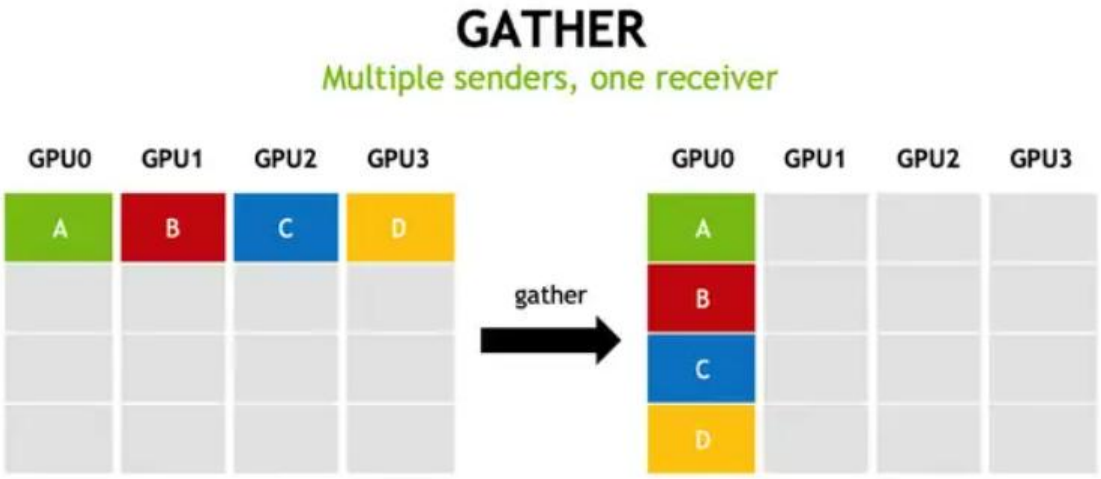

AllReduce 操作在 GPU 之间对数据（例如，求和、最小值、最大值）进行归约，并将结果存储在每个 GPU 上。

2) Reduce



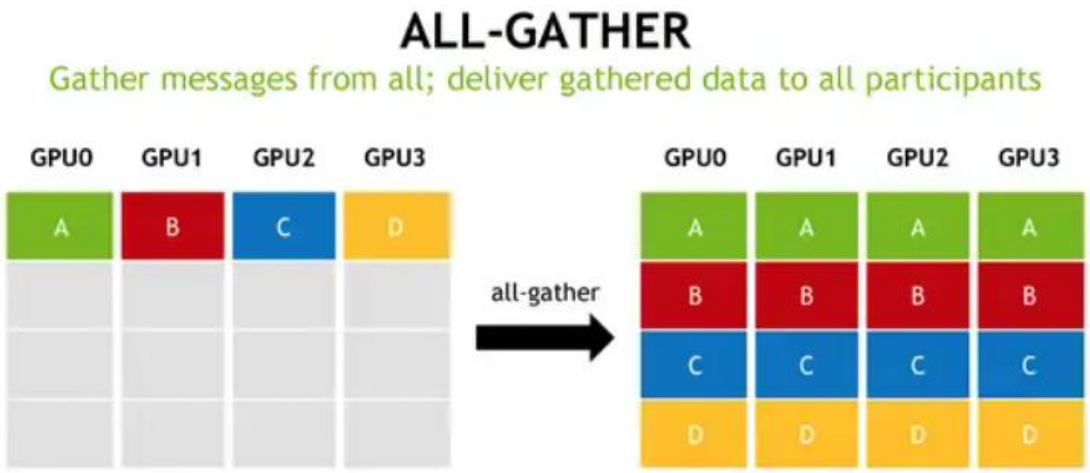

Reduce 操作执行与 AllReduce 相同的操作，但只将结果存储在指定 GPU 上。

3) Gather



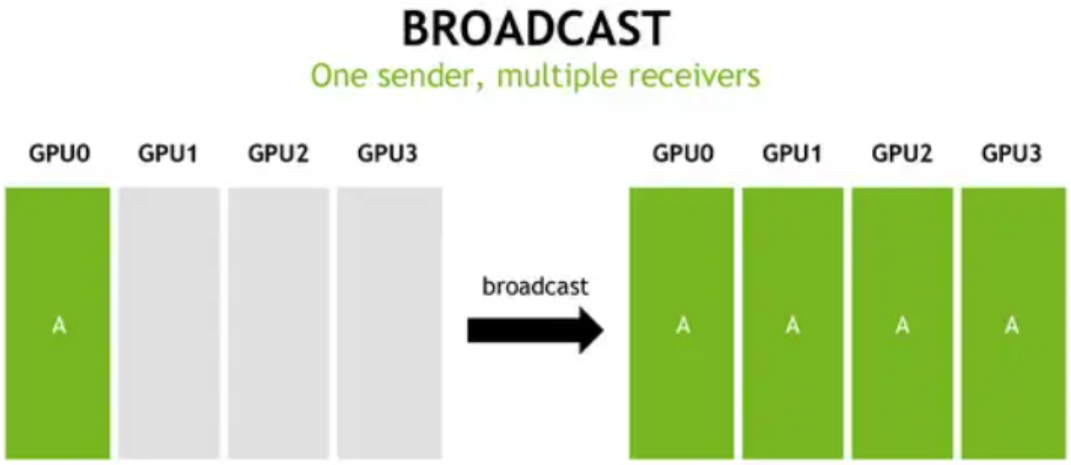

Gather 操作将多个 sender 上的数据收集到单个 GPU 上，Gather 可以理解为反向的 Scatter。

4) All-Gather



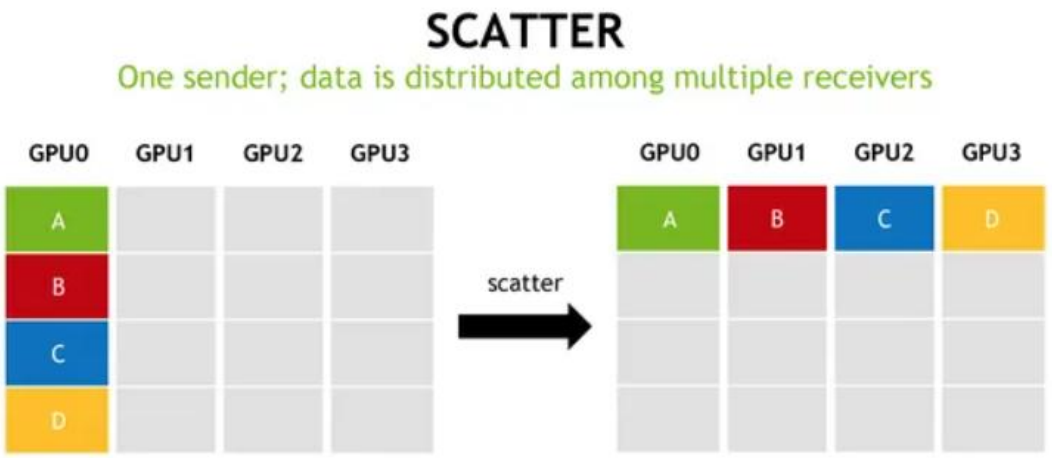

All-Gather 操作收集所有数据到所有节点上。从最基础的角度来看，All Gather 相当于一个 Gather 操作之后跟着一个 Broadcast 操作。

5) Broadcast



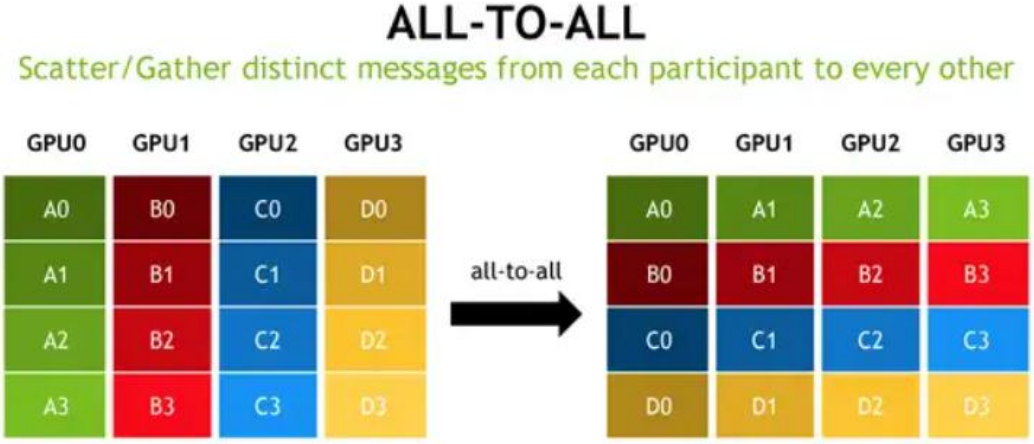

Broadcast 操作表示一个同步过程，将一张 GPU 上的信息同步给其它所有 GPU。

6) Scatter




Scatter 操作是 Gather 的反向操作，它将 GPU 的数据分散到所有 GPU。

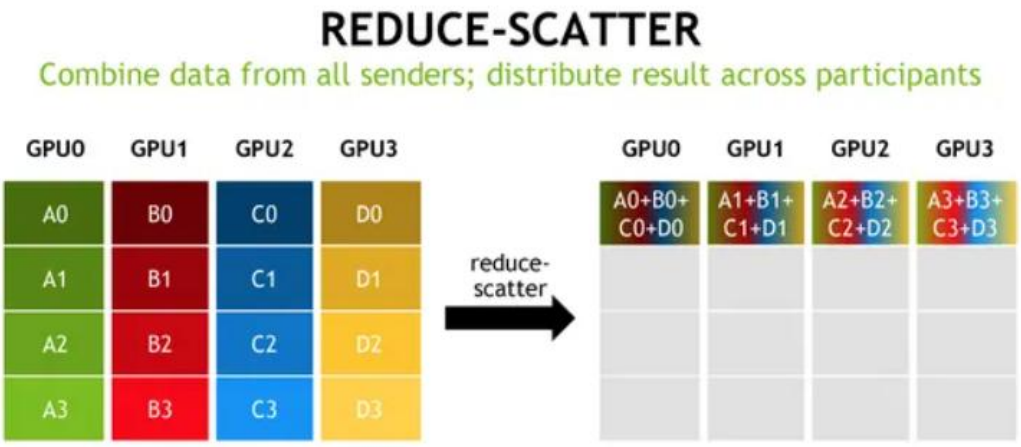
7) All-To-All



AllToAll 操作将 GPU i 的发送缓冲区中的第 j 块数据发送给 GPU j，GPU j 将接收到的来自 GPU i 的数据块放在自身的第 i 块位置。

AllToAllV 是 AllToAll 的变体，它允许每个进程发送和接收不同数量的数据。

8) Reduce-Scatter




ReduceScatter 操作执行与 Reduce 相同的操作，但结果在 GPU 之间的等大小块中分散，每个 GPU 根据其 GPU 索引获取数据块。

## 3.3 简单 demo

1) 单进程 demo

```
#include <cstdio>
#include <string>
#include <iostream>
#include <hip/hip_runtime.h>
#include <rccl.h>

#define HIP_CALL(cmd)                                   \
   do {                                                  \
      hipError_t error = (cmd);                          \
      if (error != hipSuccess)                          \
      {                                                  \
         std::cerr << "Encountered HIP error (" << hipGetErrorString(error) << ") at line
```

```
" \
                    << __LINE__ << " in file " << __FILE__ << "\n";
\
        exit(-1);
\
    }
\
  } while (0)

#define NCCL_CALL(cmd)                                  \
  do {                                                   \
    ncclResult_t error = (cmd);                          \
    if (error != ncclSuccess)                           \
    {                                                    \
      std::cerr << "Encountered NCCL error (" << ncclGetErrorString(error) << ") at
line " \
                    << __LINE__ << " in file " << __FILE__ << "\n";
\
        exit(-1);
\
    }
\
  } while (0)

int main()
{
  int numIntraRank = 4;
  ncclComm_t comm[numIntraRank];
  //初始化通信组内所有的 rank
```

```
NCCL_CALL(ncclCommInitAll(comm, numIntraRank, NULL));
// Test configuration settings
int minPow       = 10;      // Starting power of 2 input size
int maxPow       = 28;      // Ending power of 2 input size
int numIterations = 10;     // Number of timed iterations

// Allocate GPU resources for this process
hipStream_t stream[numIntraRank];
for (int i = 0; i < numIntraRank; i++)
{
   HIP_CALL(hipSetDevice(i));
   HIP_CALL(hipStreamCreate(&stream[i]));
}

// Loop over power-of-two input sizes
for (int power = minPow; power <= maxPow; power++)
{
   int N = 1 << power;

   // Allocate GPU memory
   float *iputGpu[numIntraRank], *oputGpu[numIntraRank];
   for (int r = 0; r < numIntraRank; r++)
   {
      HIP_CALL(hipSetDevice(r));
      HIP_CALL(hipMalloc((void **)&iputGpu[r], N * sizeof(float)));
      HIP_CALL(hipMalloc((void **)&oputGpu[r], N * sizeof(float)));
   }

   // Allocate CPU memory for input/output
```

```
    float *iputCpu = (float *)malloc(N * sizeof(float));
    float *oputCpu = (float *)malloc(N * sizeof(float));

    // Fill CPU memory with a simple pattern
    for (int i = 0; i < N; i++)
    {
      iputCpu[i] = 1.0f;
      oputCpu[i] = 0.0f;
    }

    // Copy the input from CPU memory to GPU memory
    for (int r = 0; r < numIntraRank; r++)
    {
      HIP_CALL(hipSetDevice(r));
      HIP_CALL(hipMemcpy(iputGpu[r], iputCpu, N * sizeof(float),
hipMemcpyHostToDevice));
    }


    NCCL_CALL(ncclGroupStart());
    for (int r = 0; r < numIntraRank; r++)
    {
      HIP_CALL(hipSetDevice(r));
      NCCL_CALL(ncclAllReduce(iputGpu[r], oputGpu[r], N, ncclFloat, ncclSum,
comm[r], stream[r]));
    }
    NCCL_CALL(ncclGroupEnd());

    for (int r = 0; r < numIntraRank; r++)
```

```
        HIP_CALL(hipStreamSynchronize(stream[r]));

    printf("finished, count: %d\n", power);

    // Release GPU resources
    for (int r = 0; r < numIntraRank; r++)
    {
        HIP_CALL(hipFree(oputGpu[r]));
        HIP_CALL(hipFree(iputGpu[r]));
    }
    free(iputCpu);
    free(oputCpu);
  }

  for (int r = 0; r < numIntraRank; r++)
  {
    HIP_CALL(hipStreamDestroy(stream[r]));
    NCCL_CALL(ncclCommDestroy(comm[r]));
  }
}
```

```
hipcc singleProcess.cpp -o singleProcess -lrccl
./singleProcess
```

2) mpi 多进程 demo

```
#include <stdio.h>
#include "hip/hip_runtime.h"
#include "rccl.h"
#include "mpi.h"
#include <unistd.h>
#include <stdint.h>
```

```
#include <stdlib.h>
#include <sys/time.h>
#include <iostream>

using namespace std;

#define MPI_CHECK(cmd) do {                                 \
    int e = cmd;                                            \
    if( e != MPI_SUCCESS ) {                                \
        printf("Failed: MPI error %s:%d '%d'\n",            \
               __FILE__,__LINE__, e);                       \
        exit(EXIT_FAILURE);                                 \
    }                                                       \
} while(0)



#define CUDA_CHECK(cmd) do {                                \
    hipError_t e = cmd;                                     \
    if( e != hipSuccess ) {                                     \
        printf("Failed: Cuda error %s:%d '%s'\n",           \
               __FILE__,__LINE__,hipGetErrorString(e));     \
        exit(EXIT_FAILURE);                                 \
    }                                                       \
} while(0)



#define NCCL_CHECK(cmd) do {                                \
    ncclResult_t r = cmd;                                   \
    if (r!= ncclSuccess) {                                  \
```

```
        printf("Failed, NCCL error %s:%d '%s'\n",                    \
            __FILE__,__LINE__,ncclGetErrorString(r));                \
        exit(EXIT_FAILURE);                                          \
    }                                                                \
} while(0)

static uint64_t getHostHash(const char* string) {
    // Based on DJB2a, result = result * 33 ^ char
    uint64_t result = 5381;
    for (int c = 0; string[c] != '\0'; c++){
        result = ((result << 5) + result) ^ string[c];
    }
    return result;
}

static void getHostName(char* hostname, int maxlen) {
    gethostname(hostname, maxlen); //本函数声明于  /usr/include/unistd.h

    for (int i=0; i< maxlen; i++) {
        if (hostname[i] == '.') {
            hostname[i] = '\0';
            return;
        }
    }
}

int main(int argc, char* argv[])
{
    if (argc != 2) {
```

```
    printf("need size\n");
    return -1;
  }
  int size = atoi(argv[1])*1024/4;
  int myRank, nRanks, localRank = 0;

  MPI_CHECK(MPI_Init(&argc, &argv));
  MPI_CHECK(MPI_Comm_rank(MPI_COMM_WORLD, &myRank));
  MPI_CHECK(MPI_Comm_size(MPI_COMM_WORLD, &nRanks)); // 初始化 MPI 环境，获取当前进程的全局 rank（myRank）和总 rank 数（nRanks）。

  uint64_t hostHashs[nRanks];
  char hostname[1024];
  getHostName(hostname, 1024);//
  hostHashs[myRank] = getHostHash(hostname);
  MPI_CHECK(MPI_Allgather(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, hostHashs, sizeof(uint64_t), MPI_BYTE, MPI_COMM_WORLD));
  //获取当前主机的名称，计算其哈希值，并将哈希值存储在 hostHashs 数组中。使用 MPI_Allgather 函数将所有进程主机哈希值收集到每个进程的 hostHashs 数组中。

  for (int p=0; p<nRanks; p++) {
    if (p == myRank) break;
    if (hostHashs[p] == hostHashs[myRank]) {
      localRank++;
    }
  }
  //遍历 hostHashs 数组，统计与当前进程主机哈希值相同的进程数量，从而计算出当前进程的本地排名 localRank，打印当前进程的全局 rank 和本地 rank。
```

```
  printf("%s: myRank = %d, localRank-- = %d\n", hostname, myRank, localRank);
  //生成本次通信唯一的通信 id（及 ip 和端口）
  ncclUniqueId id;
  ncclComm_t comm;
  float *sendbuff, *recvbuff;
  int nstream = 8;
  hipStream_t s[nstream];

  if (myRank == 0)
  {
    NCCL_CHECK(ncclGetUniqueId(&id));
  }

  //mpi 将 id 广播给所有 rank，从而使每个进程知道对应的 ip 和 port
  MPI_CHECK(MPI_Bcast((void *)&id, sizeof(id), MPI_BYTE, 0,
MPI_COMM_WORLD));

  CUDA_CHECK(hipSetDevice(localRank));// 当前进程使用本机的第 localRank
号 GPU
  CUDA_CHECK(hipMalloc(&sendbuff, size * sizeof(float)));
  CUDA_CHECK(hipMalloc(&recvbuff, size * sizeof(float)));
  for(int i = 0; i < nstream; i++){
    CUDA_CHECK(hipStreamCreate(&s[i]));
  }

  //initializing NCCL
  NCCL_CHECK(ncclCommInitRank(&comm, nRanks, id, myRank));

  //communicating using NCCL
```

```
    for (int i = 0; i < 5; i++)    {
        //start 和 end 是为了融合多个算子一同执行，此处只执行一个 allreduce
        NCCL_CHECK(ncclGroupStart());
        ncclAllReduce(sendbuff, recvbuff, size, ncclFloat, ncclSum, comm,
s[i%nstream]);
        NCCL_CHECK(ncclGroupEnd());
    }

//  同步所有被使用过的流
for(int j = 0; j < 5; j++){
        CUDA_CHECK(hipStreamSynchronize(s[j]));
}
    CUDA_CHECK(hipFree(sendbuff));
    CUDA_CHECK(hipFree(recvbuff));

    //finalizing NCCL
    ncclCommDestroy(comm);

    //finalizing MPI
    MPI_CHECK(MPI_Finalize());

    printf("[MPI Rank %d] Success \n", myRank);
    return 0;
}
```

hipcc multiProcess.cpp -o multiProcess -lmpi -lrccl

mpirun -np 8 -H host1:4,host2:4 -x LD_LIBRARY_PATH ./ multiProcess 1024

mpirun 是用于启动 MPI（Message Passing Interface）并行程序的命令，其常用参数含义如下：

-np：指定要启动的进程总数。

-H：指定要在哪些主机上启动进程，后面跟着主机名和每个主机上要启动的进程数，格式为 host1:num1,host2:num2,...。

-x：用于传递环境变量，这里 LD_LIBRARY_PATH 是动态链接库的搜索路径。

-H host1:4,host2:4：分配到两个节点，host1 分配 4 个进程槽位（slots），host2 分配 4 个进程槽位，实际分配总计 8 个槽位（4+4）与 -np 8 完全匹配。

## 3.4 细粒度通信 API 使用说明（DTK 25.04–26.04 适用）

### 3.4.1 细粒度集合通信 API 使用

集合通信的细粒度通信接口：duNcclAllreduce、duNcclAllgather、duNcclBcast、duNcclBroadcast、duNcclReduceScatter、duNcclReduce 以 duNcclAllreduce 接口为例，其细粒度控制 graph、protocol、algorithm 以及 coll nchannels 分别为 graph_test.xml、LL、TREE 以及 8 的方式如下：

```
ncclConfig_t config = NCCL_CONFIG_INITIALIZER;
config.graph = "graph_test.xml";
ncclUniqueId id;
NCCLCHECK(ncclGetUniqueId(&id));
// 带参数初始化 NCCL
NCCLCHECK(ncclGroupStart());
for (int i = 0; i < nDev; i++) {
  HIPCHECK(hipSetDevice(i));
  NCCLCHECK(ncclCommInitRankConfig(&comms[i], nDev, id, i, &config));
}
NCCLCHECK(ncclGroupEnd());


  NCCLCHECK(ncclGroupStart());
  for (int i = 0; i < nDev; ++i)
  {
  NCCLCHECK(duNcclAllReduce((const void*)sendbuff[i], (void*)recvbuff[i], size,
```

```
ncclFloat, ncclSum,comms[i], s[i],”LL”,”TREE”,8));
  }
  NCCLCHECK(ncclGroupEnd());
//synchronizing on HIP streams to wait for completion of NCCL operation  HIP 流同步
  for (int i = 0; i < nDev; ++i) {
    HIPCHECK(hipSetDevice(i));
    HIPCHECK(hipStreamSynchronize(s[i]));
  }
```

可通过配置 NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=TASK，输出 TASK 级日志，展示最终配置的算法、协议以及 nChannels 数目。

### 3.4.2 细粒度点对点通信 API 使用

点对点通信的细粒度通信接口：duNcclSend、duNcclRecv、duNcclGather、duNcclScatter、duNcclAllToAll、duNcclAllToAllv。与集合通信用例一致，唯一区别在于点对点通信时不需要配置 proto 参数。

以 duNcclGaher 接口为例，其细粒度控制 graph、protocol 以及 p2p nchannels 分别为 graph_test.xml、SIMPLE 以及 6 的方式如下：

```
ncclConfig_t config = NCCL_CONFIG_INITIALIZER;
config.graph = "graph_test.xml";
ncclUniqueId id;
NCCLCHECK(ncclGetUniqueId(&id));
// 带参数初始化 NCCL
NCCLCHECK(ncclGroupStart());
for (int i = 0; i < nDev; i++) {
  HIPCHECK(hipSetDevice(i));
NCCLCHECK(ncclCommInitRankConfig(&comms[i], nDev, id, i, &config));
}
NCCLCHECK(ncclGroupEnd());
```

```
NCCLCHECK(ncclGroupStart());
for (int i = 0; i < nDev; ++i)
{
NCCLCHECK(duNcclGather ((const void*)sendbuff[i], (void*)recvbuff[i], size, ncclFloat, root,comms[i], s[i],"Simple", 6));
}
NCCLCHECK(ncclGroupEnd());
```

# 4 API 详解

## 4.1 通讯组创建及管理类函数

### 4.1.1 ncclGetLastError

```
const char*  ncclGetLastError(ncclComm_t comm);
/*! @brief          Returns mesage on last result that occured.
    @details        Returns a human-readable message of the last error that occurred.
    @return         String containing the last result
    @param[in]      comm is currently unused and can be set to NULL */
```

### 4.1.2 ncclGetVersion

```
ncclResult_t  ncclGetVersion(int *version);
/*! @brief          Return the RCCL_VERSION_CODE of RCCL in the supplied integer.
    @details        This integer is coded with the MAJOR, MINOR and PATCH level of RCCL.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[out] version         Pointer to where version will be stored */
```

### 4.1.3 ncclGetUniqueId

```
ncclResult_t  ncclGetUniqueId(ncclUniqueId* uniqueId);
/*! @brief          Generates an ID for ncclCommInitRank.
```

```
        @details        Generates an ID to be used in ncclCommInitRank.
                        ncclGetUniqueId should be called once by a single rank and the
                        ID should be distributed to all ranks in the communicator before
                        using it as a parameter for ncclCommInitRank.
        @return         Result code. See @ref rccl_result_code for more details
        @param[out] uniqueId        Pointer to where uniqueId will be stored */
```

### 4.1.4 ncclCommInitRank

```
ncclResult_t    ncclCommInitRank(ncclComm_t* comm, int nranks, ncclUniqueId
commId, int rank);
/*! @brief          Creates a new communicator (multi thread/process version).
    @details        Rank must be between 0 and nranks-1 and unique within a
communicator clique.
                    Each rank is associated to a CUDA device, which has to be set before
calling
                    ncclCommInitRank.  ncclCommInitRank implicitly syncronizes
with other ranks,
                    so it must be called by different threads/processes or use
ncclGroupStart/ncclGroupEnd.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[out] comm            Pointer to created communicator
    @param[in]  nranks          Total number of ranks participating in this
communicator
    @param[in]  commId          UniqueId required for initialization
    @param[in]  rank            Current rank to create communicator for */
```

### 4.1.5 ncclCommInitAll

```
ncclResult_t    ncclCommInitAll(ncclComm_t* comm, int ndev, const int* devlist);
/*! @brief          Creates a clique of communicators (single process version).
    @details        This is a convenience function to create a single-process
```

communicator clique.

Returns an array of ndev newly initialized communicators in comm.

comm should be pre-allocated with size at least ndev*sizeof(ncclComm_t).

If devlist is NULL, the first ndev HIP devices are used.

Order of devlist defines user-order of processors within the communicator.

@return Result code. See @ref rccl_result_code for more details.

@param[out] comm Pointer to array of created communicators

@param[in] ndev Total number of ranks participating in this communicator

@param[in] devlist Array of GPU device indices to create for */

### 4.1.6 ncclCommInitRankConfig

ncclResult_t ncclCommInitRankConfig(ncclComm_t* comm, int nranks, ncclUniqueId commId, int rank, ncclConfig_t* config);

/*! @brief Create a new communicator with config.

@details Create a new communicator (multi thread/process version) with a configuration

set by users. See @ref rccl_config_type for more details.

Each rank is associated to a CUDA device, which has to be set before calling

ncclCommInitRank.

@return Result code. See @ref rccl_result_code for more details.

@param[out] comm Pointer to created communicator

@param[in] nranks Total number of ranks participating in this communicator

@param[in] commId UniqueId required for initialization

@param[in] rank Current rank to create communicator for. [0 to nranks-1]

```
        @param[in]    config              Pointer to communicator configuration */
```

### 4.1.7 ncclCommFinalize

```
    ncclResult_t    ncclCommFinalize(ncclComm_t comm);
    /*! @brief          Finalize a communicator.
        @details        ncclCommFinalize flushes all issued communications
                        and marks communicator state as ncclInProgress. The state will change to ncclSuccess
                        when the communicator is globally quiescent and related resources are freed; then,
                        calling ncclCommDestroy can locally free the rest of the resources (e.g. communicator
                        tself) without blocking.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]    comm                Communicator to finalize */
```

### 4.1.8 ncclCommDestroy

```
    ncclResult_t    ncclCommDestroy(ncclComm_t comm);
    /*! @brief          Frees local resources associated with communicator object.
        @details        Destroy all local resources associated with the passed in communicator object
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]    comm                Communicator to destroy */
```

### 4.1.9 ncclCommAbort

```
    ncclResult_t    ncclCommAbort(ncclComm_t comm);
    /*! @brief          Abort any in-progress calls and destroy the communicator object.
        @details        Frees resources associated with communicator object and aborts any operations
                        that might still be running on the device.
        @return         Result code. See @ref rccl_result_code for more details.
```

```
        @param[in]    comm              Communicator to abort and destroy */
```

### 4.1.10 ncclCommGetAsyncError

```
    ncclResult_t      ncclCommGetAsyncError(ncclComm_t   comm,   ncclResult_t
*asyncError);
    /*! @brief        Checks whether the comm has encountered any asynchronous
errors
        @details      Query whether the provided communicator has encountered any
asynchronous errors
        @return       Result code. See @ref rccl_result_code for more details.
        @param[in]    comm            Communicator to query
        @param[out] asyncError        Pointer to where result code will be stored */
```

### 4.1.11 ncclCommCount

```
    ncclResult_t    ncclCommCount(const ncclComm_t comm, int* count);
    /*! @brief        Gets the number of ranks in the communicator clique.
        @details      Returns the number of ranks in the communicator clique (as set
during initialization)
        @return       Result code. See @ref rccl_result_code for more details.
        @param[in]    comm            Communicator to query
        @param[out] count             Pointer to where number of ranks will be stored
*/
```

### 4.1.12 ncclCommCuDevice

```
    ncclResult_t    ncclCommCuDevice(const ncclComm_t comm, int* device);
    /*! @brief        Get the ROCm device index associated with a communicator
        @details      Returns the ROCm device number associated with the provided
communicator.
        @return       Result code. See @ref rccl_result_code for more details.
        @param[in]    comm            Communicator to query
        @param[out] device            Pointer to where the associated ROCm device
```

```
index will be stored */
```

### 4.1.13 ncclCommUserRank

```
    ncclResult_t  ncclCommUserRank(const ncclComm_t comm, int* rank);
    /*! @brief          Get the rank associated with a communicator
        @details        Returns the user-ordered "rank" associated with the provided communicator.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]  comm            Communicator to query
        @param[out] rank            Pointer to where the associated rank will be stored */
```

### 4.1.14 ncclCommShrink

```
    ncclResult_t  ncclCommShrink(ncclComm_t comm, int* excludeRanksList, int excludeRanksCount, ncclComm_t* newcomm, ncclConfig_t* config, int shrinkFlags);
    /*! @brief          Shrink existing communicator.
     @details        Ranks in excludeRanksList will be removed form the existing communicator.
                     Within the new communicator, ranks will be re-ordered to fill the gap of removed ones.
                     If config is NULL, the new communicator will inherit the original communicator's configuration.
                     The flag enables NCCL to adapt to various states of the parent communicator, see NCCL_SHRINK flags.
     @return         Result code. See @ref rccl_result_code for more details.

     @param[in]  comm                    Original communicator object for this rank
     @param[in]  excludeRanksList        List of ranks to be exluded
     @param[in]  excludeRanksCount       Number of ranks to be excluded
```

@param[out] newcomm Pointer to new communicator

@param[in] config Config file for new communicator. May be NULL to inherit from comm

@param[in] shrinkFlags Flag to adapt to various states of the parent communicator (see NCCL_SHRINK flags)*/

### 4.1.15 ncclCommInitRankScalable

ncclResult_t ncclCommInitRankScalable(ncclComm_t* newcomm, int nranks, int myrank, int nId, ncclUniqueId* commIds, ncclConfig_t* config);

/*! @brief Creates a new communicator (multi thread/process version), similar to ncclCommInitRankConfig.

@details Allows to use more than one ncclUniqueId (up to one per rank), indicated by nId, to accelerate the init operation.

The number of ncclUniqueIds and their order must be the same for every rank.

@return Result code. See @ref rccl_result_code for more details.

@param[out] newcomm Pointer to new communicator

@param[in] nranks Total number of ranks participating in this communicator

@param[in] myrank Current rank

@param[in] nId Number of unique IDs

@param[in] commIds List of unique IDs

@param[in] config Config file for new communicator. May be NULL to inherit from comm */

## 4.2 集合通讯函数

### 4.2.1 ncclAllReduce

ncclResult_t ncclAllReduce(const void* sendbuff, void* recvbuff, size_t count, ncclDataType_t datatype, ncclRedOp_t op, ncclComm_t comm,

```
hipStream_t stream);
    /*! @brief          All-Reduce
        @details        Reduces data arrays of length *count* in *sendbuff* using *op*
operation, and
                        leaves identical copies of result on each *recvbuff*.
                        In-place operation will happen if sendbuff == recvbuff.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]    sendbuff          Input data array to reduce
        @param[out] recvbuff            Data array to store reduced result array
        @param[in]    count             Number of elements in data buffer
        @param[in]    datatype          Data buffer element datatype
        @param[in]    op                Reduction operator
        @param[in]    comm              Communicator group object to execute on
        @param[in]    stream            HIP stream to execute collective on */
```

### 4.2.2 ncclBroadcast

```
    ncclResult_t    ncclBroadcast(const void* sendbuff, void* recvbuff, size_t count,
ncclDataType_t datatype, int root,
                ncclComm_t comm, hipStream_t stream);
    /*! @brief          Broadcast
        @details        Copies *count* values from *sendbuff* on *root* to *recvbuff*
on all devices.
                        *root* is the rank (not the HIP device) where data resides before
the operation is started.
                       *sendbuff* may be NULL on ranks other than *root*.
                        In-place operation will happen if *sendbuff* == *recvbuff*.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]    sendbuff          Data array to copy (if *root*).    May be NULL
for other ranks
        @param[in]    recvbuff          Data array to store received array
```

```
        @param[in]  count           Number of elements in data buffer
        @param[in]  datatype        Data buffer element datatype
        @param[in]  root            Rank of broadcast root
        @param[in]  comm            Communicator group object to execute on
        @param[in]  stream          HIP stream to execute collective on  */
```

### 4.2.3 ncclReduce

```
    ncclResult_t  ncclReduce(const void* sendbuff, void* recvbuff, size_t count,
ncclDataType_t datatype,
            ncclRedOp_t op, int root, ncclComm_t comm, hipStream_t stream);
    /*! @brief          Reduce
        @details        Reduces data arrays of length *count* in *sendbuff* into
*recvbuff* using *op*operation.
                        *recvbuff* may be NULL on all calls except for root device.
                        *root* is the rank (not the HIP device) where data will reside
after the
                        operation is complete.
                        In-place operation will happen if sendbuff == recvbuff.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]  sendbuff        Local device data buffer to be reduced
        @param[out] recvbuff        Data buffer where result is stored (only for
*root* rank).  May be null for other ranks.
        @param[in]  count           Number of elements in every send buffer
        @param[in]  datatype        Data buffer element datatype
        @param[in]  op              Reduction operator type
        @param[in]  root            Rank where result data array will be stored
        @param[in]  comm            Communicator group object to execute on
        @param[in]  stream          HIP stream to execute collective on  */
```

### 4.2.4 ncclAllGather

```
ncclResult_t    ncclAllGather(const void* sendbuff, void* recvbuff, size_t sendcount,
                ncclDataType_t datatype, ncclComm_t comm, hipStream_t stream);
    /*! @brief          All-Gather
        @details        Each device gathers *sendcount* values from other GPUs into *recvbuff*,
                        receiving data from rank i at offset i*sendcount.
                        Assumes recvcount is equal to nranks*sendcount, which means that recvbuff
                        should have a size of at least nranks*sendcount elements.
                        In-place operations will happen if sendbuff == recvbuff + rank * sendcount.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]      sendbuff        Input data array to send
        @param[out]     recvbuff        Data array to store the gathered result
        @param[in]      sendcount       Number of elements each rank sends
        @param[in]      datatype        Data buffer element datatype
        @param[in]      comm            Communicator group object to execute on
        @param[in]      stream          HIP stream to execute collective on    */
```

### 4.2.5 ncclReduceScatter

```
    ncclResult_t    ncclReduceScatter(const void* sendbuff, void* recvbuff,
                size_t recvcount, ncclDataType_t datatype, ncclRedOp_t op, ncclComm_t comm,hipStream_t stream);
    /*! @brief          Reduce-Scatter
        @details        Reduces data in *sendbuff* using *op* operation and leaves reduced result
                        scattered over the devices so that *recvbuff* on rank i will contain the i-th
```

```
                        block of the result.
                        Assumes sendcount is equal to nranks*recvcount, which means
that *sendbuff*
                        should have a size of at least nranks*recvcount elements.
                        In-place operations will happen if recvbuff == sendbuff + rank *
recvcount.
        @return         Result code. See @ref rccl_result_code for more details.
        @param[in]   sendbuff          Input data array to reduce
        @param[out]   recvbuff         Data array to store reduced result subarray
        @param[in]    recvcount        Number of elements each rank receives
        @param[in]    datatype         Data buffer element datatype
        @param[in]    op               Reduction operator
        @param[in]    comm              Communicator group object to execute on
        @param[in]    stream           HIP stream to execute collective on */
```

### 4.2.6 ncclAllReduceWithBias

```
    ncclResult_t    ncclAllReduceWithBias(const void* sendbuff, void* recvbuff,
size_t count,
     ncclDataType_t datatype, ncclRedOp_t op, ncclComm_t comm, hipStream_t
stream, const void* acc);
    /*! @brief          All-Reduce-with-Bias
     @details       Reduces data arrays of length *count* in *sendbuff* using *op*
operation, and
                        leaves identical copies of result on each *recvbuff*.
                        In-place operation will happen if sendbuff == recvbuff.
     @return        Result code. See @ref rccl_result_code for more details.

     @param[in]    sendbuff          Input data array to reduce
     @param[out] recvbuff          Data array to store reduced result array
     @param[in]    count              Number of elements in data buffer
```

```
    @param[in]  datatype        Data buffer element datatype
    @param[in]  op              Reduction operator
    @param[in]  comm            Communicator group object to execute on
    @param[in]  stream          HIP stream to execute collective on
    @param[in]  acc             Bias data array to reduce */
```

## 4.3 组调用函数

### 4.3.1 ncclGroupStart

```
    ncclResult_t  ncclGroupStart();
    /*! @brief        Group Start
        @details      Start a group call. All calls to RCCL until ncclGroupEnd will be fused into
                      a single RCCL operation. Nothing will be started on the HIP stream until
                      ncclGroupEnd.
        @return       Result code. See @ref rccl_result_code for more details. */
```

### 4.3.2 ncclGroupEnd

```
    ncclResult_t  ncclGroupEnd();
    /*! @brief        Group End
        @details      End a group call. Start a fused RCCL operation consisting of all calls since
                      ncclGroupStart. Operations on the HIP stream depending on the RCCL operations
                      need to be called after ncclGroupEnd.
        @return       Result code. See @ref rccl_result_code for more details. */
```

### 4.3.3 ncclGroupSimulateEnd

```
    ncclResult_t  ncclGroupSimulateEnd(ncclSimInfo_t* simInfo);
    /*! @brief            Simulate a ncclGroupEnd call and report the estimated communication time.
```

```
    @details    Ends the current group like ncclGroupEnd does, but in addition to enqueuing
                the fused operations it returns the simulation information in *simInfo*. On
                return, the *estimatedTime* field of *simInfo* holds the time RCCL estimates
                for the operations that were submitted inside the group, which allows an
                application to compare several communication strategies without executing
                them. The same usage constraints as ncclGroupEnd apply.
                *simInfo* must be initialized with NCCL_SIM_INFO_INITIALIZER before the call.
    @return     Result code. See @ref rccl_result_code for more details.

    @param[in,out] simInfo  Simulation information, must be initialized with
NCCL_SIM_INFO_INITIALIZER */
```

## 4.4 点对点通讯函数

### 4.4.1 ncclSend

```
    ncclResult_t ncclSend(const void* sendbuff, size_t count, ncclDataType_t datatype, int peer,
                          ncclComm_t comm, hipStream_t stream);
    /*! @brief      Send
        @details    Send data from *sendbuff* to rank *peer*.
                    Rank *peer* needs to call ncclRecv with the same *datatype* and the same *count*
                    as this rank.
                    This operation is blocking for the GPU. If multiple ncclSend and
```

```
ncclRecv operations
                need to progress concurrently to complete, they must be fused within a ncclGroupStart /
                 ncclGroupEnd section.
        @return       Result code. See @ref rccl_result_code for more details.
@param[in]  sendbuff        Data array to send
        @param[in]  count           Number of elements to send
        @param[in]  datatype        Data buffer element datatype
        @param[in]  peer            Peer rank to send to
        @param[in]  comm            Communicator group object to execute on
        @param[in]  stream          HIP stream to execute collective on */
```

### 4.4.2 ncclRecv

```
    ncclResult_t  ncclRecv(void* recvbuff, size_t count, ncclDataType_t datatype, int peer,
                ncclComm_t comm, hipStream_t stream);
    /*! @brief          Receive
        @details        Receive data from rank *peer* into *recvbuff*.
                        Rank *peer* needs to call ncclSend with the same datatype and the same count
                        as this rank.
                        This operation is blocking for the GPU. If multiple ncclSend and ncclRecv operations
                        need to progress concurrently to complete, they must be fused within a ncclGroupStart/
                        ncclGroupEnd section.
        @return       Result code. See @ref rccl_result_code for more details.
        @param[out] recvbuff        Data array to receive
        @param[in]  count           Number of elements to receive
        @param[in]  datatype        Data buffer element datatype
```

```
        @param[in]    peer            Peer rank to send to
        @param[in]    comm            Communicator group object to execute on
        @param[in]    stream          HIP stream to execute collective on    */
```

## 4.5 数据类型

### 4.5.1 ncclComm_t

```
    typedef struct ncclComm* ncclComm_t;
    /*! @brief          Opaque handle to communicator
        @details        A communicator contains information required to facilitate collective communications calls */
```

### 4.5.2 ncclResult_t

```
    typedef enum {
            ncclSuccess                 =  0, /*!< No error */
            ncclUnhandledCudaError      =  1, /*!< Unhandled HIP error */
            ncclSystemError             =  2, /*!< Unhandled system error */
            ncclInternalError           =  3, /*!< Internal Error - Please report to RCCL developers */
            ncclInvalidArgument         =  4, /*!< Invalid argument */
            ncclInvalidUsage            =  5, /*!< Invalid usage */
            ncclRemoteError             =  6, /*!< Remote process exited or there was a network error */
            ncclInProgress              =  7, /*!< RCCL operation in progress */
            ncclNumResults              =  8  /*!< Number of result types */
    } ncclResult_t;
    /*! @brief          Result type
        @details        Return codes aside from ncclSuccess indicate that a call has failed */
```

### 4.5.3 ncclDataType_t

```
typedef enum { ncclInt8             = 0, ncclChar           = 0,
               ncclUint8            = 1,
               ncclInt32            = 2, ncclInt            = 2,
               ncclUint32           = 3,
               ncclInt64            = 4,
               ncclUint64           = 5,
               ncclFloat16          = 6, ncclHalf           = 6,
               ncclFloat32          = 7, ncclFloat          = 7,
               ncclFloat64          = 8, ncclDouble         = 8,
               ncclBfloat16         = 9,
               ncclFloat8e4m3       = 10,
               ncclFloat8e5m2       = 11,
               ncclNumTypes         = 12 } ncclDataType_t;
/*! @brief           Data types
    @details         Enumeration of the various supported datatype */
```

### 4.5.4 ncclRedOp_t

```
typedef enum { ncclSum              = 0, /*!< Sum */
               ncclProd             = 1, /*!< Product */
               ncclMax              = 2, /*!< Max */
               ncclMin              = 3, /*!< Min */
               ncclAvg              = 4, /*!< Average */
               ncclNumOps           = 5, /*!< Number of built-in reduction ops */
               ncclMaxRedOp         = 0x7fffffff>>(32-
8*sizeof(ncclRedOp_dummy_t)) /*!< Largest value for ncclRedOp_t */
             } ncclRedOp_t;
/*! @brief          Reduction operation selector
    @details        Enumeration used to specify the various reduction operations
```

```
                    ncclNumOps is the number of built-in ncclRedOp_t values and
serves as
                     the least possible value for dynamic ncclRedOp_t values
constructed by
                     ncclRedOpCreate functions.
                    ncclMaxRedOp is the largest valid value for ncclRedOp_t and is
defined
                    to be the largest signed value (since compilers are permitted to
use
                    signed enums) that won't grow sizeof(ncclRedOp_t) when
compared to previous
                    RCCL versions to maintain ABI compatibility. */
```

### 4.5.5 ncclScalarResidence_t

```
    typedef enum {
      ncclScalarDevice              = 0, /*!< Scalar is in device-visible memory */
       ncclScalarHostImmediate    = 1    /*!< Scalar is in host-visible memory */
} ncclScalarResidence_t;
/*! @brief          Location and dereferencing logic for scalar arguments.
    @details        Enumeration specifying memory location of the scalar argument.
                    Based on where the value is stored, the argument will be
dereferenced either
                    while the collective is running (if in device memory), or before the
ncclRedOpCreate()
                    function returns (if in host memory). */
```

### 4.5.6 ncclConfig_t

```
    typedef struct ncclConfig_v22700 {
  /* attributes that users should never touch. */
  size_t size;                          /*!< Should not be touched */
```

```
  unsigned int magic;                /*!< Should not be touched */
  unsigned int version;              /*!< Should not be touched */
  /* attributes that users are able to customize. */
  int blocking;                      /*!< Whether or not calls should block or not */
  int cgaClusterSize;                /*!< Cooperative group array cluster size */
  int minCTAs;                       /*!< Minimum number of cooperative thread arrays (blocks) */
  int maxCTAs;                       /*!< Maximum number of cooperative thread arrays (blocks) */
  const char *netName;               /*!< Force NCCL to use a specfic network */
  int splitShare;                    /*!< Allow communicators to share resources */
  int trafficClass;                  /*!< Traffic class*/
  const char *commName;              /*!< Name of the communicator*/
  int collnetEnable;                 /*!< Check for collnet enablement*/
  int CTAPolicy;                     /*!< CTA Policy*/
  int shrinkShare;                   /*!< Shrink size*/
  int nvlsCTAs;                      /*!< Number of NVLS cooperative thread arrays (blocks)*/
} ncclConfig_t;
/*! @brief          Communicator configuration
    @details        Users can assign value to attributes to specify the behavior of a communicator */
```

### 4.5.7 ncclSimInfo_t

```
    typedef struct ncclSimInfo_v22200 {
     /* attributes that users should never touch. */
     size_t size;                    /*!< Should not be touched */
     unsigned int magic;             /*!< Should not be touched */
     unsigned int version;           /*!< Should not be touched */
     /* attributes that users are able to read after the simulation. */
```

```
    float estimatedTime;          /*!< Estimated time in us for the operations submitted
in the group */
} ncclSimInfo_t;
/*! @brief          Simulation information
    @details       Structure that reports the result of the simulation performed by
                   ncclGroupSimulateEnd. It must be initialized with
                   NCCL_SIM_INFO_INITIALIZER before it is passed to
ncclGroupSimulateEnd. */

/* NCCL_SIM_INFO_INITIALIZER must be assigned to initialize simInfo structure
when it is created.
 * Not initialized simInfo will result in NCCL error. */
#define NCCL_SIM_INFO_INITIALIZER {           \
  sizeof(ncclSimInfo_t),     /* size */          \
  0x74685283,                /* magic */         \
  NCCL_VERSION(NCCL_MAJOR, NCCL_MINOR, NCCL_PATCH), /* version
*/   \
  NCCL_UNDEF_FLOAT   /* estimated time */      \
}
```

## 4.6 自定义归约操作符

### 4.6.1 ncclRedOpCreatePreMulSum

```
    ncclResult_t     ncclRedOpCreatePreMulSum(ncclRedOp_t *op, void *scalar,
ncclDataType_t datatype, ncclScalarResidence_t residence, ncclComm_t comm);
    /*! @brief          Create a custom pre-multiplier reduction operator
        @details       Creates a new reduction operator which pre-multiplies input
values by a given
                       scalar locally before reducing them with peer values via
summation. For use
```

```
                only with collectives launched against *comm* and *datatype*. The
                *residence* argument indicates how/when the memory pointed to by *scalar*
                will be dereferenced. Upon return, the newly created operator's handle
                is stored in *op*.
    @return     Result code. See @ref rccl_result_code for more details.
    @param[out] op          Pointer to where newly created custom reduction operator is to be stored
    @param[in]  scalar      Pointer to scalar value.
    @param[in]  datatype    Scalar value datatype
    @param[in]  residence   Memory type of the scalar value
    @param[in]  comm        Communicator to associate with this custom reduction operator */
```

### 4.6.2 ncclRedOpDestroy

```
ncclResult_t ncclRedOpDestroy(ncclRedOp_t op, ncclComm_t comm);
/*! @brief      Destroy custom reduction operator
    @details    Destroys the reduction operator *op*. The operator must have been created by
                ncclRedOpCreatePreMul with the matching communicator *comm*. An operator may be
                destroyed as soon as the last RCCL function which is given that operator returns.
    @return     Result code. See @ref rccl_result_code for more details
    @param[in]  op          Custom reduction operator is to be destroyed
    @param[in]  comm        Communicator associated with this reduction operator */
```

## 4.7 细粒度通信接口（DTK 25.04–26.04 适用）

### 4.7.1 duNcclAllreduce

```
    ncclResult_t    duNcclAllreduce(const void* sendbuff, void* recvbuff, size_t count,
            ncclDataType_t datatype, ncclRedOp_t op, ncclComm_t comm, hipStream_t
stream,
            const char* proto = nullptr, const char* algo = nullptr, int nChannels = 0);
/*! @brief          All-Reduce with user config
    @details        Reduces data arrays of length *count* in *sendbuff* using *op*
operation, and
                    leaves identical copies of result on each *recvbuff*.
                    In-place operation will happen if sendbuff == recvbuff.
                    Use "proto","algo","nChannels" to adjust the Protocol,Algorithm
and
                    nChannels during actual communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[in]    sendbuff          Input data array to reduce
    @param[out] recvbuff            Data array to store reduced result array
    @param[in]    count             Number of elements in data buffer
    @param[in]    datatype          Data buffer element datatype
    @param[in]    op                Reduction operator
    @param[in]    comm              Communicator group object to execute on
    @param[in]    stream            HIP stream to execute collective on
    @param[in]    proto             protocol configured by the user
    @param[in]    algo              Algorithm configured by the user
    @param[in]    nChannels         nChannels configured by the user */
```

### 4.7.2 duNcclAllgather

```
ncclResult_t    duNcclAllgather(const void* sendbuff, void* recvbuff, size_t sendcount,
            ncclDataType_t datatype, ncclComm_t comm, hipStream_t stream,
```

```
        const char* proto = nullptr, const char* algo = nullptr, int nChannels = 0);
/*! @brief          All-Gather with user config
    @details        Each device gathers *sendcount* values from other GPUs into *recvbuff*,
                    receiving data from rank i at offset i*sendcount.
                    Assumes recvcount is equal to nranks*sendcount, which means that recvbuff
                    should have a size of at least nranks*sendcount elements.
                    In-place operations will happen if sendbuff == recvbuff + rank * sendcount.
                    Use "proto","algo","nChannels" to adjust the Protocol,Algorithm and
                    nChannels during actual communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[in]  sendbuff        Input data array to send
    @param[out] recvbuff        Data array to store the gathered result
    @param[in]  sendcount       Number of elements each rank sends
    @param[in]  datatype        Data buffer element datatype
    @param[in]  comm            Communicator group object to execute on
    @param[in]  stream          HIP stream to execute collective on
    @param[in]  proto           protocol configured by the user
    @param[in]  algo            Algorithm configured by the user
    @param[in]  nChannels       nChannels configured by the user */
```

### 4.7.3 duNcclBcast

```
ncclResult_t    duNcclBcast(void* buff, size_t count, ncclDataType_t datatype, int root,
        ncclComm_t comm, hipStream_t stream, const char* proto = nullptr, const char* algo = nullptr, int nChannels = 0);
/*! @brief          (Deprecated) Broadcast (in-place) with user config
    @details        Copies *count* values from *root* to all other devices.
```

root is the rank (not the CUDA device) where data resides before the operation is started.

This operation is implicitly in-place.

Use "proto","algo","nChannels" to adjust the Protocol,Algorithm and

nChannels during actual communication.

@return Result code. See @ref rccl_result_code for more details.

@param[in,out] buff Input array on *root* to be copied to other ranks. Output array for all ranks.

@param[in] count Number of elements in data buffer

@param[in] datatype Data buffer element datatype

@param[in] root Rank owning buffer to be copied to others

@param[in] comm Communicator group object to execute on

@param[in] stream HIP stream to execute collective on

@param[in] proto protocol configured by the user

@param[in] algo Algorithm configured by the user

@param[in] nChannels nChannels configured by the user */

### 4.7.4 duNcclBroadcast

```
ncclResult_t duNcclBroadcast(const void* sendbuff, void* recvbuff, size_t count, ncclDataType_t datatype, int root,
        ncclComm_t comm, hipStream_t stream, const char* proto = nullptr, const char* algo = nullptr, int nChannels = 0);
```

/*! @brief Broadcast with user config

@details Copies *count* values from *sendbuff* on *root* to *recvbuff* on all devices.

*root* is the rank (not the HIP device) where data resides before the operation is started.

*sendbuff* may be NULL on ranks other than *root*.

In-place operation will happen if *sendbuff* == *recvbuff*.

```
                Use "proto","algo","nChannels" to adjust the Protocol,Algorithm and
                nChannels during actual communication.
    @return     Result code. See @ref rccl_result_code for more details.
    @param[in]  sendbuff    Data array to copy (if *root*). May be NULL for other ranks
    @param[in]  recvbuff    Data array to store received array
    @param[in]  count       Number of elements in data buffer
    @param[in]  datatype    Data buffer element datatype
    @param[in]  root        Rank of broadcast root
    @param[in]  comm        Communicator group object to execute on
    @param[in]  stream      HIP stream to execute collective on
    @param[in]  proto       protocol configured by the user
    @param[in]  algo        Algorithm configured by the user
    @param[in]  nChannels   nChannels configured by the user */
```

### 4.7.5 duNcclReduceScatter

```
ncclResult_t  duNcclReduceScatter(const void* sendbuff, void* recvbuff,
        size_t recvcount, ncclDataType_t datatype, ncclRedOp_t op, ncclComm_t comm,
        hipStream_t stream, const char* proto = nullptr, const char* algo = nullptr, int nChannels = 0);
/*! @brief      Reduce-Scatter with user config
    @details    Reduces data in *sendbuff* using *op* operation and leaves reduced result
                scattered over the devices so that *recvbuff* on rank i will contain the i-th
                block of the result.
                Assumes sendcount is equal to nranks*recvcount, which means that *sendbuff*
```

```
                    should have a size of at least nranks*recvcount elements.
                    In-place operations will happen if recvbuff == sendbuff + rank *
recvcount.
                    Use "proto","algo","nChannels" to adjust the Protocol,Algorithm
and
                    nChannels during actual communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[in]  sendbuff        Input data array to reduce
    @param[out] recvbuff        Data array to store reduced result subarray
    @param[in]  recvcount       Number of elements each rank receives
    @param[in]  datatype        Data buffer element datatype
    @param[in]  op              Reduction operator
    @param[in]  comm            Communicator group object to execute on
    @param[in]  stream          HIP stream to execute collective on
    @param[in]  proto           protocol configured by the user
    @param[in]  algo            Algorithm configured by the user
    @param[in]  nChannels       nChannels configured by the user */
```

### 4.7.6 duNcclReduce

```
ncclResult_t  duNcclReduce(const void* sendbuff, void* recvbuff, size_t count,
ncclDataType_t datatype,
            ncclRedOp_t op, int root, ncclComm_t comm, hipStream_t stream,
            const char* proto = nullptr, const char* algo = nullptr, int nChannels = 0);
/*! @brief          Reduce with user config
    @details        Reduces data arrays of length *count* in *sendbuff* into *recvbuff*
using *op*
                    operation.
                    *recvbuff* may be NULL on all calls except for root device.
                    *root* is the rank (not the HIP device) where data will reside after
the
```

```
                operation is complete.
                In-place operation will happen if sendbuff == recvbuff.
                Use "proto","algo","nChannels" to adjust the Protocol,Algorithm and
                nChannels during actual communication.
    @return      Result code. See @ref rccl_result_code for more details.
    @param[in]  sendbuff      Local device data buffer to be reduced
    @param[out] recvbuff      Data buffer where result is stored (only for *root* rank).  May be null for other ranks.
    @param[in]  count         Number of elements in every send buffer
    @param[in]  datatype      Data buffer element datatype
    @param[in]  op            Reduction operator type
    @param[in]  root          Rank where result data array will be stored
    @param[in]  comm          Communicator group object to execute on
    @param[in]  stream        HIP stream to execute collective on
    @param[in]  proto         protocol configured by the user
    @param[in]  algo          Algorithm configured by the user
    @param[in]  nChannels     nChannels configured by the user */
```

### 4.7.7 duNcclSend

```
ncclResult_t  duNcclSend(const void* sendbuff, size_t count, ncclDataType_t datatype, int peer,
        ncclComm_t comm, hipStream_t stream, const char* proto = nullptr, int nChannels = 0);
/*! @brief       Send with user config
    @details     Send data from *sendbuff* to rank *peer*.
                 Rank *peer* needs to call ncclRecv with the same *datatype* and the same *count*
                 as this rank.
                 This operation is blocking for the GPU. If multiple ncclSend and
```

```
ncclRecv operations
                need to progress concurrently to complete, they must be fused within
a ncclGroupStart /
                ncclGroupEnd section.
                Use "proto","nChannels" to adjust the Protocol and p2pnChannels
during actual p2p communication.
    @return      Result code. See @ref rccl_result_code for more details.
    @param[in]   sendbuff      Data array to send
    @param[in]   count          Number of elements to send
    @param[in]   datatype       Data buffer element datatype
    @param[in]   peer           Peer rank to send to
    @param[in]   comm           Communicator group object to execute on
    @param[in]   stream         HIP stream to execute collective on
    @param[in]   proto          protocol configured by the user
    @param[in]   nChannels      nChannels configured by the user */
```

### 4.7.8 duNcclRecv

```
ncclResult_t  duNcclRecv(void* recvbuff, size_t count, ncclDataType_t datatype, int
peer,
          ncclComm_t comm, hipStream_t stream, const char* proto = nullptr, int
nChannels = 0);
/*! @brief        Receive with user config
    @details      Receive data from rank *peer* into *recvbuff*.
                  Rank *peer* needs to call ncclSend with the same datatype and the
same count
                  as this rank.
                  This operation is blocking for the GPU. If multiple ncclSend and
ncclRecv operations
                  need to progress concurrently to complete, they must be fused within
a ncclGroupStart/
```

```
                    ncclGroupEnd section.
                    Use "proto","nChannels" to adjust the Protocol and p2pnChannels
during actual p2p communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[out] recvbuff        Data array to receive
    @param[in]  count           Number of elements to receive
    @param[in]  datatype        Data buffer element datatype
    @param[in]  peer            Peer rank to send to
    @param[in]  comm            Communicator group object to execute on
    @param[in]  stream          HIP stream to execute collective on
    @param[in]  proto           protocol configured by the user
    @param[in]  nChannels       nChannels configured by the user */
```

### 4.7.9 duNcclGather

```
ncclResult_t    duNcclGather(const void* sendbuff, void* recvbuff, size_t sendcount,
            ncclDataType_t datatype, int root, ncclComm_t comm, hipStream_t stream,
            const char* proto = nullptr, int nChannels = 0);
/*! @brief          Gather with user config
    @details        Root device gathers *sendcount* values from other GPUs into
*recvbuff*,
                    receiving data from rank i at offset i*sendcount.
                    Assumes recvcount is equal to nranks*sendcount, which means that
*recvbuff*
                    should have a size of at least nranks*sendcount elements.
                    In-place operations will happen if sendbuff == recvbuff + rank *
sendcount.
                    *recvbuff* may be NULL on ranks other than *root*.
                    Use "proto","nChannels" to adjust the Protocol and p2pnChannels
during actual p2p communication.
    @return         Result code. See @ref rccl_result_code for more details.
```

```
    @param[in]    sendbuff        Data array to send
    @param[out] recvbuff          Data array to receive into on *root*.
    @param[in]    sendcount       Number of elements to send per rank
    @param[in]    datatype         Data buffer element datatype
    @param[in]    root            Rank that receives data from all other ranks
    @param[in]    comm            Communicator group object to execute on
    @param[in]    stream          HIP stream to execute collective on
    @param[in]    proto           protocol configured by the user
    @param[in]    nChannels       nChannels configured by the user */
```

### 4.7.10 duNcclScatter

```
ncclResult_t    duNcclScatter(const void* sendbuff, void* recvbuff,
            size_t recvcount, ncclDataType_t datatype, int root, ncclComm_t comm,
            hipStream_t stream, const char* proto = nullptr, int nChannels = 0);
/*! @brief          Scatter with user config
    @details        Scattered over the devices so that recvbuff on rank i will contain the
i-th
                    block of the data on root.
                    Assumes sendcount is equal to nranks*recvcount, which means that
*sendbuff*
                    should have a size of at least nranks*recvcount elements.
                    In-place operations will happen if recvbuff == sendbuff + rank *
recvcount.
                    Use "proto","nChannels" to adjust the Protocol and p2pnChannels
during actual p2p communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[in]    sendbuff        Data array to send (on *root* rank).    May be NULL
on other ranks.
    @param[out] recvbuff          Data array to receive partial subarray into
    @param[in]    recvcount       Number of elements to receive per rank
```

```
    @param[in]    datatype          Data buffer element datatype
    @param[in]    root              Rank that scatters data to all other ranks
    @param[in]    comm              Communicator group object to execute on
    @param[in]    stream            HIP stream to execute collective on
    @param[in]    proto             protocol configured by the user
    @param[in]    nChannels         nChannels configured by the user */
```

### 4.7.11 duNcclAllToAll

```
ncclResult_t    duNcclAllToAll(const void* sendbuff, void* recvbuff, size_t count,
            ncclDataType_t datatype, ncclComm_t comm, hipStream_t stream, const
char* proto = nullptr, int nChannels = 0);
/*! @brief          All-To-All with user config
    @details        Device (i) send (j)th block of data to device (j) and be placed as (i)th
                    block. Each block for sending/receiving has *count* elements,
which means
                    that *recvbuff* and *sendbuff* should have a size of nranks*count
elements.
                    In-place operation is NOT supported. It is the user's responsibility
                    to ensure that sendbuff and recvbuff are distinct.
                    Use "proto","nChannels" to adjust the Protocol and p2pnChannels
during actual p2p communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[in]    sendbuff          Data array to send (contains blocks for each other
rank)
    @param[out] recvbuff            Data array to receive (contains blocks from each
other rank)
    @param[in]    count             Number of elements to send between each pair of
ranks
    @param[in]    datatype          Data buffer element datatype
    @param[in]    comm              Communicator group object to execute on
```

```
    @param[in]  stream          HIP stream to execute collective on
    @param[in]  proto           protocol configured by the user
    @param[in]  nChannels       nChannels configured by the user */
```

### 4.7.12 duNcclAllToAllv

```
ncclResult_t  duNcclAllToAllv(const void *sendbuff, const size_t sendcounts[],
        const size_t sdispls[], void *recvbuff, const size_t recvcounts[],
        const size_t rdispls[], ncclDataType_t datatype, ncclComm_t comm,
hipStream_t stream,
        const char* proto = nullptr, int nChannels = 0);
/*! @brief          All-To-Allv
    @details        Device (i) sends sendcounts[j] of data from offset sdispls[j]
                    to device (j). At the same time, device (i) receives recvcounts[j] of
data
                    from device (j) to be placed at rdispls[j].
                    sendcounts, sdispls, recvcounts and rdispls are all measured in the
units
                    of datatype, not bytes.
                    In-place operation will happen if sendbuff == recvbuff.
                    Use "proto","nChannels" to adjust the Protocol and p2pnChannels
during actual p2p communication.
    @return         Result code. See @ref rccl_result_code for more details.
    @param[in]  sendbuff        Data array to send (contains blocks for each other
rank)
    @param[in]  sendcounts      Array containing number of elements to send to each
participating rank
    @param[in]   sdispls              Array of offsets into *sendbuff* for each
participating rank
    @param[out] recvbuff        Data array to receive (contains blocks from each
other rank)
```

```
    @param[in]    recvcounts        Array containing number of elements to receive
from each participating rank
    @param[in]    rdispls           Array of offsets into *recvbuff* for each
participating rank
    @param[in]    datatype          Data buffer element datatype
    @param[in]    comm              Communicator group object to execute on
    @param[in]    stream            HIP stream to execute collective on
    @param[in]    proto             protocol configured by the user
    @param[in]    nChannels         nChannels configured by the user */
```

## 4.8 内存管理

### 4.8.1 ncclMemAlloc

```
    ncclResult_t    ncclMemAlloc(void** ptr, size_t size);
    /*! @brief          Allocate device memory for RCCL.
     @details       Allocates *size* bytes of device memory and returns the address of
the
                    allocated block in *ptr*. The memory is intended for the RCCL
                    optimizations that require a dedicated allocation (e.g. user buffer
                    registration and symmetric memory windows) and is aligned to the
RCCL
                    registration granularity, so the actual allocated size might be larger
                    than requested. When unified memory management is enabled
                    (NCCL_CUMEM_ENABLE=1), the block is allocated through the
VMM allocator so
                    that it can be mapped for peer access and registered afterwards;
otherwise
                    a plain device allocation is performed.
     @return        Result code. See @ref rccl_result_code for more details.
```

```
    @param[out] ptr                 Pointer to where the address of the allocated
memory will be stored
    @param[in]  size                Number of bytes to allocate */
```

### 4.8.2 ncclMemFree

```
    ncclResult_t  ncclMemFree(void *ptr);
    /*! @brief        Free device memory allocated by ncclMemAlloc.
    @details      Releases the memory block pointed to by *ptr*, which must have
been
                  returned by ncclMemAlloc. Both the VMM allocation path and the
plain device
                  allocation path are handled. The device that was current before the
call is
                  restored before returning.
    @return       Result code. See @ref rccl_result_code for more details.

    @param[in]  ptr               Pointer to the memory block to free */
```

### 4.8.3 ncclCommRegister

```
    ncclResult_t  ncclCommRegister(const ncclComm_t comm, void* buff, size_t
size, void** handle);
    /*! @brief        Register a user buffer with a communicator.
    @details      Registers the buffer [*buff*, *buff* + *size*) with *comm* so that
RCCL can
                  reuse the registration for the following communications instead of
                  repeating the setup for every call, which enables zero-copy
operation on the
                  registered range. An opaque handle of the registration is returned
in
                  *handle* and must be given back to ncclCommDeregister once the
```

```
buffer is no
                longer used. Registrations are reference counted per communicator,
so a
                buffer registered several times requires one deregister call per
                registration.
                Registration is skipped, and *handle* is set to NULL, when local
buffer
                registration is disabled (NCCL_LOCAL_REGISTER=0). Buffers
allocated as
                managed memory cannot be registered with MSCCL++; RCCL
user buffer
                registration is used for them instead.
    @return     Result code. See @ref rccl_result_code for more details.

    @param[in]  comm        Communicator to register the buffer with
    @param[in]  buff        Pointer to the buffer to register
    @param[in]  size        Size of the buffer in bytes
    @param[out] handle      Pointer to where the opaque registration handle will
be stored */
```

### 4.8.4 ncclCommDeregister

```
    ncclResult_t  ncclCommDeregister(const ncclComm_t comm, void* handle);
    /*! @brief                Deregister a buffer previously registered with
ncclCommRegister.
    @details    Releases the registration identified by *handle* on *comm*. Since
                registrations are reference counted, the buffer is only really
unregistered
                after the last matching deregister call. ncclInvalidUsage is returned
when
                *handle* is not a valid handle of *comm*.
```

```
    @return         Result code. See @ref rccl_result_code for more details.

    @param[in]  comm            Communicator the buffer was registered with
    @param[in]  handle          Handle returned by ncclCommRegister */
```

### 4.8.5 ncclCommWindowRegister

```
    ncclResult_t  ncclCommWindowRegister(ncclComm_t comm, void* buff, size_t
size, ncclWindow_t* win, int winFlags);
    /*! @brief          Register a memory window on a communicator.
        @details        Registers the buffer [*buff*, *buff* + *size*) with *comm* and
returns a
                        window descriptor in *win* that identifies the registered range
for the
                        symmetric memory collectives. The buffer must belong to an
allocation whose
                        base address and size are aligned to the RCCL registration page
size, and
                        the registered range must stay inside that allocation.
*winFlags* selects
                        the behavior of the window: NCCL_WIN_DEFAULT, or
NCCL_WIN_COLL_SYMMETRIC to
                        make the window usable by collective symmetric memory
kernels.
                        When local registration or unified memory management is
disabled
                        (NCCL_LOCAL_REGISTER=0                          or
NCCL_CUMEM_ENABLE=0), the call returns success
                        without creating a window and *win* is set to NULL.
        @return         Result code. See @ref rccl_result_code for more details.
```

```
        @param[in]  comm            Communicator to register the window with
        @param[in]  buff            Pointer to the buffer used as a communication
window
        @param[in]  size            Size of the buffer in bytes
        @param[out] win             Pointer to where the window descriptor will
be stored
        @param[in]  winFlags        Window registration flags, see the
NCCL_WIN flags */
```

### 4.8.6 ncclCommWindowDeregister

```
    ncclResult_t  ncclCommWindowDeregister(ncclComm_t comm, ncclWindow_t
win);
    /*! @brief          Deregister a memory window previously registered with
ncclCommWindowRegister.
        @details        Releases the symmetric memory mapping associated with
*win* and frees the
                        window descriptor. A NULL window is accepted, in which
case the call is a
                        no-op.
        @return         Result code. See @ref rccl_result_code for more details.

        @param[in]  comm            Communicator the window was registered
with
        @param[in]  win                     Window descriptor returned by
ncclCommWindowRegister */
```

# 5 性能调优

## 5.1 拓扑感知配置

RCCL 获取节点内拓扑的方式有三种，优先级由高到低为：1）使用环境变量

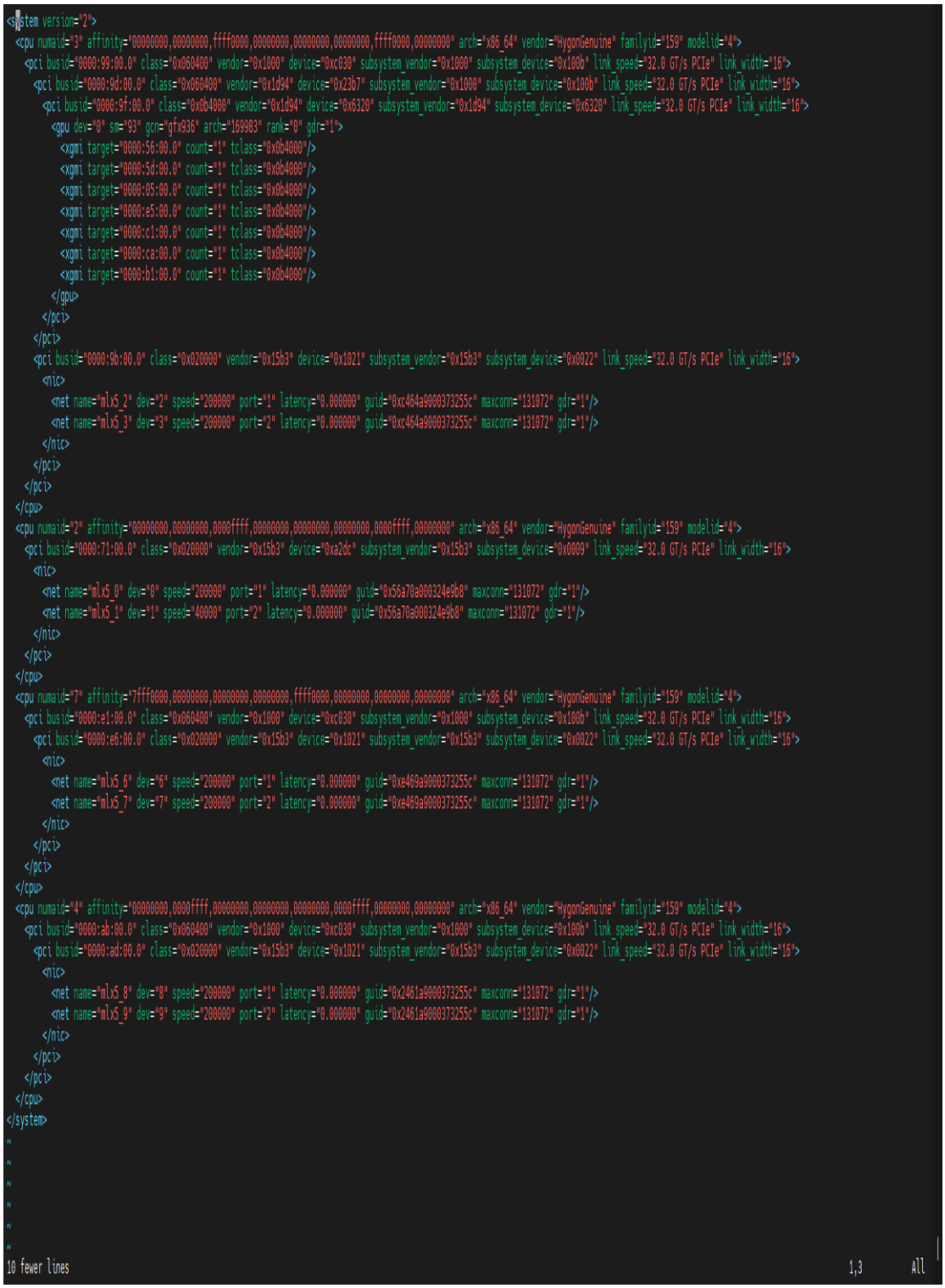
NCCL_TOPO_FILE 指定的自定义拓扑文件；2）使用 RCCL 自动探查生成并缓存的拓扑文件（默认路径为/tmp/topo.xml，可通过 RCCL_TOPO_FILE_NAME 修改）；3）在既未指定自定义拓扑文件、也不存在缓存文件（或是指定 RCCL_FORCE_DETECT_TOPO=1）时，由 RCCL 在初始化阶段自动探查硬件拓扑。

拓扑自动探查功能会按照网卡及 DCU 的 PCIeSwitch 匹配关系及 OAM ID 配对等系统信息自动探查每块 DCU 对应的最优网卡集合，并在指定目录下生成 topo 文件，供后续 graph 搜索阶段使用。探查结果会按文件路径缓存在本地，硬件拓扑（如更换网卡、调整 pcie 槽位）发生变化后不会自动刷新，需删除缓存文件（默认/tmp/topo.xml）或设置 RCCL_FORCE_DETECT_TOPO=1 重新探查。

## 5.2 拓扑文件生成

生成的拓扑可以使用环境变量 NCCL_TOPO_DUMP_FILE 指定一个文件路径，保存系统硬件拓扑信息。生成的拓扑包含的设备主要包括：GPU、PCI（pcie 插槽）、NVS（xgmi）、CPU（numa）、NIC（可以理解为网卡卡槽）、NET（具体网卡类型如 ib、roce 等）

开发者社区




## 5.3 环境变量优化

RCCL 内部众多参数有着默认配置，应用启动前可以根据特定场景设置多个环境变量进行自定义配置，常用的环境变量如：使用 NCCL_PROTO 指定通信协议，NCCL_ALGO 指定算法，NCCL_IB_HCA 指定网卡，以及 channel 相关环境

变量设置。

以下表格为常用的环境变量及其解释：

| | 环境变量 | 常用值 | 含义 |
|---|---|---|---|
| 1 | NCCL_DEBUG | INFO WARN TRACE | 控制调试信息输出级别 |
| 2 | NCCL_PROTO | LL SIMPLE | 指定集合操作的通信协议 |
| 3 | NCCL_ALGO | RING TREE | 调整集合操作的算法实现。 |
| 4 | NCCL_IB_HCA | mlx5_x:1,mlx5_y:1 | 显示指定 RCCL 中使用的 HCA 设备 |
| 5 | NCCL_NET_GDR_READ | 0 1 | 指定是否启用 GDR 来加速读取操作，1 为开启 |
| 6 | NCCL_NET_GDR_LEVEL | 0 1 2 3 4 5 | 控制 GDR 的支持级别，5 为完全支持 |
| 7 | NCCL_TOPO_FILE | topo-input.xml | 指定一个自定义的拓扑文件，供RCCL加载系统硬件拓扑信息。 |
| 8 | NCCL_TOPO_DUMP_FILE | topo-output.xml | 指定一个文件 |

| | 环境变量 | 常用值 | 含义 |
| --- | --- | --- | --- |
| | | | 路径，用于保存RCCL 自动检测到的系统硬件拓扑信息 |
| 9 | NCCL_GRAPH_FILE | graph-input.xml | 指定一个自定义的通信图文件，供RCCL 加载通信图。 |
| 10 | NCCL_GRAPH_DUMP_FILE | graph-output.xml | 指定一个文件路径，用于保存RCCL 自动生成的通信图。 |
| 11 | NCCL_MIN_P2P_NCHANNELS | 2 4 8 16 32 | P2P 通信最小通道数 |
| 12 | NCCL_MAX_P2P_NCHANNELS | 2 4 8 16 32 | P2P 通信最大通道数 |
| 13 | NCCL_NCHANNELS_PER_PEER | 2 4 8 16 32 | 控制直接通信的GPU 分配多少通道 |
| 14 | NCCL_MIN_NCHANNELS | 2 4 8 16 32 | 控制所有集合通信的最小通道数 |
| 15 | NCCL_MAX_NCHANNELS | 2 4 8 16 32 | 控制所有集合通信的最大通 |

| | 环境变量 | 常用值 | 含义 |
|---|---|---|---|
| | | | 道数 |
| 16 | NCCL_P2P_LL_THRESHOLD | 8192 | p2p 低延迟协议的阈值，当数据量小于该阈值时，使用低延迟协议 |
| 17 | NCCL_P2P_DISABLE | 0 1 | 禁用 P2P 通信，默认 0 启用 P2P 通信 |
| 18 | NCCL_SHM_DISABLE | 0 1 | 禁用共享内存通信，默认 0 启用 SHM 通信 |
| 19 | NCCL_P2P_LEVEL | SYS PXB NVL | 设置 P2P 通信的支持级别，默认 ALL 允许所有 GPU 使用 P2P |
| 20 | NCCL_NET | libibverbs.so | 指定网络通信插件，默认使用 InfiniBand Verbs 库 |
| 21 | NCCL_NET_PLUGIN | libnccl-net-custom.so | 指定自定义的网络通信插件路径 |
| 22 | NCCL_COLLNET_ENABLE | 0 1 | 启用或禁用集 |

| | 环境变量 | 常用值 | 含义 |
|---|---|---|---|
| | | | 合通信网络 |
| 23 | NCCL_LAUNCH_MODE | GROUP PARALLEL | 设置RCCL的启动模式，默认GROUP |
| 24 | NCCL_HOSTID | 一个整数表示节点的唯一ID | 多节点通信指定当前节点的唯一标识符 |
| 25 | NCCL_COMM_ID | 一个字符串表示通信任务的唯一ID | 多节点通信指定通信任务的唯一标识符 |
| 26 | NCCL_RINGS | 一个字符串用于定义通信环的GPU顺序 | 手动指定GPU的通信环拓扑，仅适用于单节点内的GPU通信 |
| 27 | NCCL_GDRCOPY_ENABLE | 0 1 | 控制是否启用GDRCOPY技术，默认为0禁用 |
| 28 | NCCL_DEBUG_SUBSYS | init,coll,graph 等可以多个一起使用 | 指定特定模块的日志输出 |
| 29 | NCCL_SOCKET_IFNAME | eth0 ib0 | 指定通信使用的网络接口 |

| | 环境变量 | 常用值 | 含义 |
| --- | --- | --- | --- |
| 30 | NCCL_DEBUG_FILE | %h-%p.log | 将日志文件按照主机名-进程号.log 的格式输出到文件中 |
| | RCCL_SIMPLE_CHANNELS | 2 4 8 16 32 | 设置集合通信 Simple 协议使用的通道数 |
| | RCCL_COLL_XHCL_CHANNEL_NUM | 2 4 8 16 32 | 集合通信中使用常规显存缓冲区的通道数，其余通道使用扩展 PCIe 内存通道 |
| | RCCL_P2P_XHCL_CHANNEL_NUM | 2 4 8 16 32 | 点对点通信中使用常规显存缓冲区的通道数，其余通道使用扩展 PCIe 内存通道 |
| | NCCL_LOCAL_REGISTER | 0 1（默认 1） | 用户缓冲区注册（零拷贝）总开关，置 0 时注册与内存窗口注册均不生效 |

| | 环境变量 | 常用值 | 含义 |
| --- | --- | --- | --- |
| | NCCL_PASS_SM | 0 1（默认 0） | 是否启用 SM-free 通信模式，仅在单节点或 P2P 网络关闭时生效 |

## 5.4 通道数、算法、通信协议配置

通道数配置：RCCL 优先使用环境变量设置的通道数值，可以通过 NCCL_MIN_P2P_NCHANNELS 、 NCCL_MAX_P2P_NCHANNELS 、 NCCL_NCHANNELS_PER_PEER 、 NCCL_MIN_NCHANNELS 、 NCCL_MAX_NCHANNELS 设置通道数，

目前 RCCL 内部会根据不同的环境默认配置不同的通道数，如检测到当前环境支持 hylink 同时用户没有自定义 channel 相关环境变量，则将 channel 相关的环境变量都设置为 32。

算法配置：目前 RCCL 使用两种算法，ring 算法和 tree 算法，可以使用 NCCL_ALGO 指定算法。

- ring 算法：

特点：1）带宽效率高，每个 GPU 同时收发数据，充分利用双向带宽；2）延迟和 GPU 数量线性相关，通信步骤数为环的大小（即 GPU 数量），单机内延迟较低，但跨节点时扩展性受限。

适用场景：适用于单机多 GPU 等中小规模集群。

- tree 算法：

特点：1）拓展性高，延迟和树的高度相关，扩展性优于 Ring；2）带宽存在瓶颈风险；3）单节点或小规模集群场景下性能可能不如 ring 算法；

适用场景：适合大规模的跨节点集群。

通信协议配置：目前 RCCL 使用两种通讯协议：LL、Simple 协议，可以使用

NCCL_PROTO 指定通信协议。

- LL（Low Latency）协议：

优势：利用原子操作避免了 Simple 协议中所 memory fence 操作，降低了通信延迟；

劣势：由于传输时需要传输 Flag，因此有效带宽仅占 50%；

适用场景：LL 协议不适用于大数据量传输的情况，适用于延迟敏感的小 size 场景。

- Simple 协议：

优势：没有额外的数据传输，因此有效带宽可达到 100%；

劣势：使用 memory fence 操作会导致延迟增加；

适用场景：适用于大数据量且对延迟不敏感的场景，不适用于要求低延迟的情况；

汇总（延迟为 DCU 场景下，供参考）：

| 协议 | 带宽利用率 | 延迟 |
| --- | --- | --- |
| LL | 50% | ~1us |
| Simple | 100% | ~6us |

## 5.5 超节点支持

当前版本支持超节点场景使用，在初始化阶段通过识别通信组内的全部 DCU 是否处于同一超节点互联域，确认超节点场景，并根据识别结果调整显存映射方式、通道与内存路径的划分、P2P 通道数、协议参数与算子分块等内容。通过对超节点环境进行的针对性性能调优，使得 RCCL 能充分利用超节点内的高带宽互联能力。

开发者社区

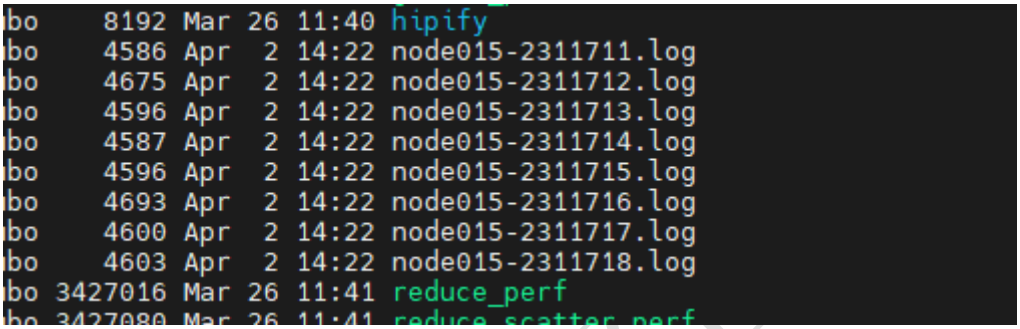
# 6 调试与诊断

## 6.1 日志分析

### 6.1.1 保存日志进行确认

添加以下环境变量导出日志：NCCL_DEBUG=info NCCL_DEBUG_SUBSYS=all NCCL_DEBUG_FILE=%h-%p.log

这样可以将日志文件按照 主机名-进程号.log 的格式输出到文件中。



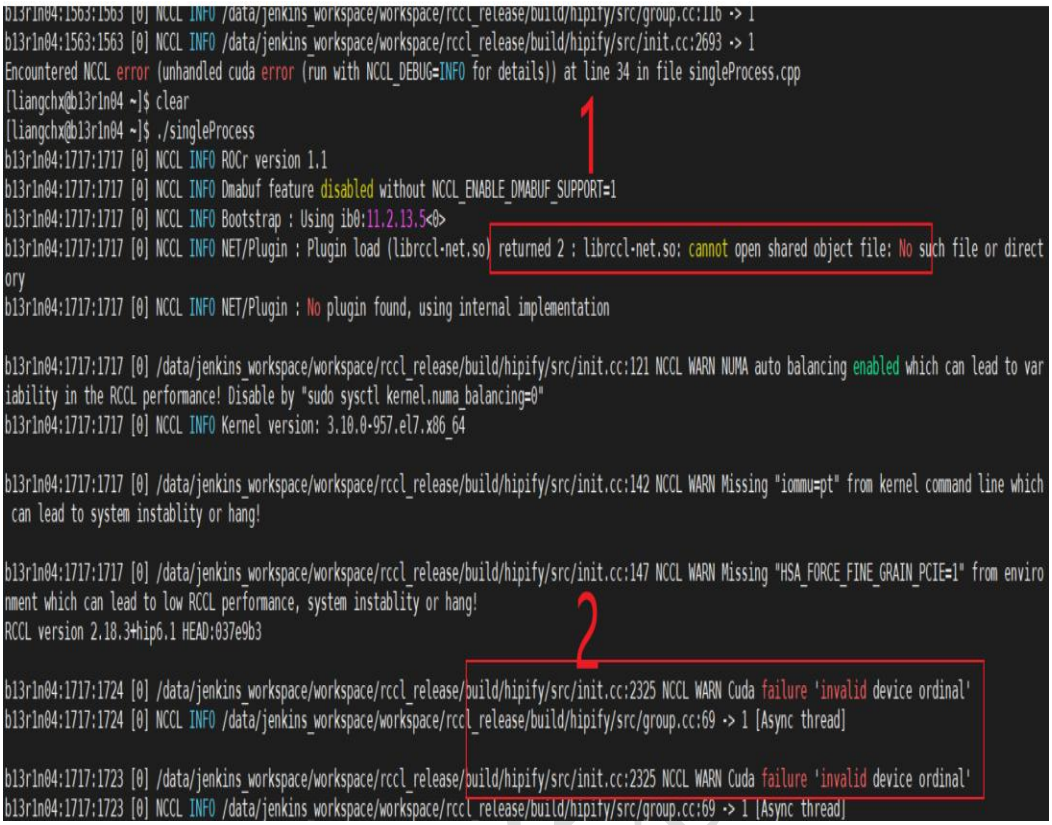

卡住问题进一步导出 pstack 信息，建议按照进程号命名线程栈信息文件，上一步的文件名中可以找到每个机器上的进程号（有条件的话可以放 debug 库上去跑，pstack 信息更全）。

pstack 2311711 > 2311711.log，这样就可以得到每个 rank 的日志和线程栈信息用于分析。若遇到复现时间较长，日志量很大的情况可能有几十 G，同时还掺杂其它组件的日志，可能分析比较困难，这时候最好抓取 NCCL 关键字重定向到新的文件，然后取文件的最后 100000 行（具体多少行可灵活控制）进行分析。

grep --line-buffered 'NCCL' test.log | tail -n 100000 > nccl.log

### 6.1.2 报错问题日志分析

INFO 级别的日志一般不是问题所在，WARN 级别的日志可能是问题所在，下图 1 不是问题所在，2 才是问题所在。




## 6.2 hipprof 分析

hipprof 参数--rccl-trace：用户开启 RCCL API、核函数的跟踪分析以及开启 HIP API 以及核函数、memcpy 的跟踪分析，可视化页面中核函数的详细信息增加带宽、数据量等显示。

RCCL 接口跟踪分为单节点和多节点两类。以 RCCL_TESTS 为例进行演示。

单节点执行指令：hipprof --rccl-trace ./all_reduce_perf -b 8 -e 1G -f 2 -g 4

执行成功后，除 HIP 接口的统计表和核函数的统计表外，还可在终端显示 RCCL 算子接口统计表和 RCCL 核函数的统计表。

HIP PROF:rccl API statistics

| Name | Calls | TotalDurationNs | AverageNs | Percentage |
|---|---|---|---|---|
| ncclAllReduce | 4968 | 12122296 | 2440 | 100.0 |
| Total | 4968 | 12122296 | 2440 | 100.0 |

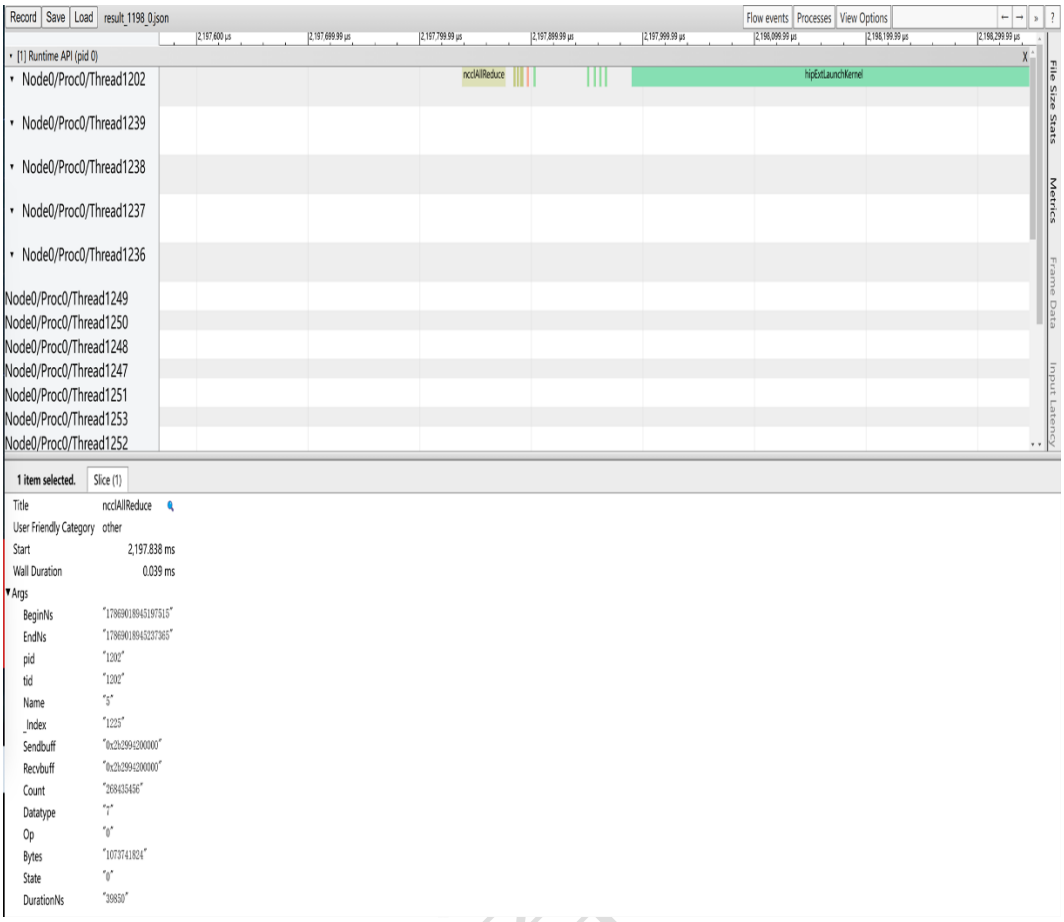
HIP PROF:rccl kernel statistics

| Name | Pars | Calls | TotalDurationNs | AverageNs | TotalBytes | TotalBWGB/s | AverageBWGB/s | Percentage |
|---|---|---|---|---|---|---|---|---|
| ncclKernel_AllReduce_Ring_Simp... | (1,1,4),(1,1,256) ... | 2484 | 150655707492 | 60650445 | 39940888985... | 5820.453478 | 2.343178 | 99.930264 |
| ncclKernel_AllReduce_Ring_LL_P... | (1,1,1),(1,1,256) ... | 1956 | 76140128 | 38926 | 2882336 | 67.68586 | 0.034604 | 0.050504 |
| ncclKernel_AllReduce_Ring_LL_P... | (1,1,4),(1,1,256) ... | 352 | 20428540 | 58035 | 17301504 | 277.997021 | 0.789764 | 0.01355 |
| ncclKernel_AllReduce_Ring_LL_P... | (1,1,2),(1,1,256) ... | 176 | 8565656 | 48668 | 2883584 | 58.275621 | 0.331111 | 0.005682 |
| Total | | 4968 | 150760841816 | 30346385 | 39943195728... | 6224.411980 | 1.252901 | 100.0 |

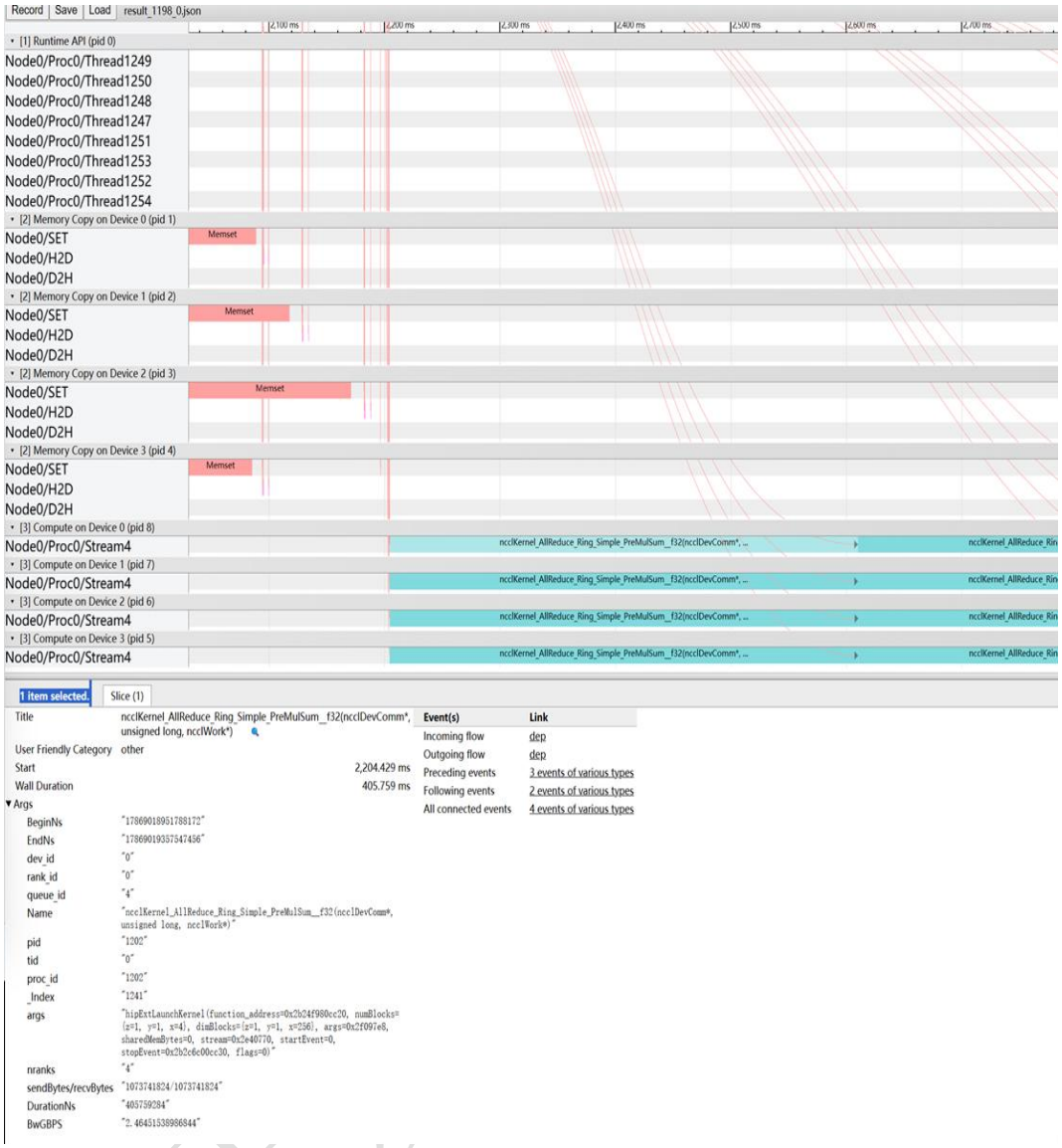
以上表格除了输出在终端外会另外形成两个 csv 格式的文件：result_1198.rccltrace.csv 和 result_1198.rcclkernel.csv。文件名称中 1198 是本次性能分析的进程号，将文件下载到本地后可以使用 Excel 等支持 csv 的图表工具打开。

性能分析过程还会产生一个 json 格式的文件：result_1198.json，把文件下载到本地后在 chrome 浏览器中输入 chrome://tracing 打开可视化工具，点击 load 按钮，选择下载好的 json 文件。可视化结果如下：

开发者社区


Record
Save
Load
result_1198_0.json
Flow events
Processes
View Options
[1] Runtime API (pid 0)
Node0/Proc0/Thread1202
Node0/Proc0/Thread1239
Node0/Proc0/Thread1238
Node0/Proc0/Thread1237
Node0/Proc0/Thread1236
Node0/Proc0/Thread1249
Node0/Proc0/Thread1250
Node0/Proc0/Thread1248
Node0/Proc0/Thread1247
Node0/Proc0/Thread1251
Node0/Proc0/Thread1253
Node0/Proc0/Thread1252
ncclAllReduce
hipExtLaunchKernel
File Size Stats
Metrics
Frame Data
Input Latency
1 item selected.
Slice (1)
Title ncclAllReduce
User Friendly Category other
Start 2,197.838 ms
Wall Duration 0.039 ms
Args
BeginNs "1786901894519751 5"
EndNs "1786901894523273 65"
pid "1202"
tid "1202"
Name "S"
_Index "1225"
Sendbuff "0x2b2994200000"
Recvbuff "0x2b2994200000"
Count "268435456"
Datatype "7"
Op "0"
Bytes "1073741824"
State "0"
DurationNs "39850"

开发者




左边目录结构没有变化，Runtime API 中发生了一些变化，增加了 RCCL API 的 timeline 显示。

在多节点多进程的 RCCL 程序性能分析过程需要两个步骤：

1. 每个进程启动一个 hipprof 分别对每个进程进行分析，生成数据库文件，这里需要把 mpirun 放在 hipprof 前面，再由 hipprof 来启动 RCCL 程序。
2. 将多个数据库文件进行合并，生成合并后的可视化脚本。

以 rccl-tests 为例，mpi 开启两个节点，每个节点 4 个进程：

runtest.sh:

```
#!/bin/bash
```

```
source ${ROCM_PATH}/env.sh
hipprof –rccl-trace NCCL_IB_HCA=mlx5_0:1
${RCCL_TEST}/build/all_reduce_perf -b 8 -e 10M -f 2 -g 1
```

mpirun -np 8 -H b13r2n01:4,b13r3n05:4 sh runtest.sh

执行 hipprof --db-merge ./ 将所有 db 文件汇总成一个 db 文件导出 json，若需要在合并 db 导出新 json 时打印 RCCL 统计信息，执行如下：

hipprof --rccl-trace --db-merge ./

```
[zhengjj@h03r1n06 rccl-tests]$ hipprof --rccl-trace --db-merge ./
HIP_PROF:create new db file './hip-prof-3184.db'
HIP_PROF:merge file ./hip-prof-31648.db.
HIP_PROF:merge file ./hip-prof-31652.db.
HIP_PROF:merge file ./hip-prof-31655.db.
HIP_PROF:merge file ./hip-prof-1841.db.
HIP_PROF:merge file ./hip-prof-1842.db.
HIP_PROF:merge file ./hip-prof-31656.db.
HIP_PROF:merge file ./hip-prof-1845.db.
HIP_PROF:merge file ./hip-prof-1846.db.
--------------------------------------------------------------------------------------------------------------------------------------------------------
HIP PROF:rccl API statistics
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Name                                                |Calls         |TotalDurationNs          |AverageNs                |Percentage                |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|ncclAllReduce                                       |6416          |4218412                  |657                      |100.0                     |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Total                                               |6416          |4218412                  |657                      |100.0                     |
--------------------------------------------------------------------------------------------------------------------------------------------------------

--------------------------------------------------------------------------------------------------------------------------------------------------------
HIP PROF:rccl kernel statistics
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Name                          |Pars                |Calls   |TotalDurationNs   |AverageNs   |TotalBytes   |TotalBWGBps   |AverageBWGBps   |Percentage   |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|ncclKernel_AllReduce_Ring_Simp...|(1,1,4),(1,1,256) ...|1096    |522005648         |476282      |687865856    |1567.15097    |1.429882        |56.417088    |
|ncclKernel_AllReduce_Tree_LL_P...|(1,1,1),(1,1,256) ...|2856    |169173216         |59234       |718400       |15.426892     |0.005402        |18.283826    |
|ncclKernel_AllReduce_Tree_Simp...|(1,1,1),(1,1,256) ...|1408    |111013014         |78844       |10813440     |143.801631    |0.102132        |11.998014    |
|ncclKernel_AllReduce_Tree_Simp...|(1,1,4),(1,1,256) ...|704     |90380837          |128381      |69206016     |542.017232    |0.769911        |9.768139     |
|ncclKernel_AllReduce_Tree_Simp...|(1,1,2),(1,1,256) ...|352     |32688883          |92866       |11534336     |129.916134    |0.36908         |3.532934     |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Total                         |                    |6416    |925261598         |144211      |780138048    |2398.312859   |0.373802        |100.0        |
--------------------------------------------------------------------------------------------------------------------------------------------------------


--------------------------------------------------------------------------------------------------------------------------------------------------------
HIP PROF:hip API statistics
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Name                                                |Calls         |TotalDurationNs          |AverageNs                |Percentage                |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|hipMalloc                                           |128           |3179264481               |24838003                 |31.1511403369797          |
|hipHostMalloc                                       |368           |2875626952               |7814203                  |28.1760323099565          |
|hipFuncGetAttributes                                |112           |1778542773               |15879846                 |17.426557572718           |
|hipDeviceSynchronize                                |864           |611499192                |707753                   |5.99160505827123          |
|hipExtMallocWithFlags                               |120           |459289621                |3827413                  |4.50022183577158          |
|hipMemsetAsync                                      |208           |429736476                |2066040                  |4.21065354952301          |
|hipStreamDestroy                                    |128           |365974407                |2859175                  |3.58589861910891          |
|hipStreamQuery                                      |147855        |201322878                |1361                     |1.97260632548885          |
|hipLaunchKernel                                     |1440          |81628026                 |56686                    |0.799809549835506         |
|hipExtLaunchKernel                                  |6416          |51858386                 |8082                     |0.50811999743637          |
|hipMemcpyAsync                                      |552           |39693896                 |71909                    |0.3889296194787           |
```

# 保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区